Machine Learning based intrusion detection system

Elzobeir Salih Mahmoud Mohamad
Afrah Hamed Ali Noor Mohammed

BACHELOR OF COMPUTER SCIENCE
(HONORS)
ALBUKHARY INTERNATIONAL UNIVERSITY
JANUARY 2025

Machine Learning based intrusion detection system

Elzobeir Salih Mahmoud Mohamad (AIU21102130)
Afrah Hamed Ali Noor Mohammed (AIU21102155)

Report Submitted to Fulfill the Partial Requirements for Bachelor of Computer Science (Honours)

DECLARATION

We declare that this dissertation is our original work and all references have been cited adequately as required.

Date: 3 FEBRUARY 2025	Signature	: …….
	Full name	: Elzobeir Salih Mahmoud Mohamad
	ID Number	: AIU 22102144
Date: 3 FEBRUARY 2025	Signature	:…….
	Full name	:Afrah Hamed Ali Noor Mohammed
	ID Number	: AIU21102155

APPROVAL

This thesis has been approved in partial fulfillment for the Bachelor of Computer Science (Honors)

Supervisor	Name: Professor Dr. Khalid Hussain Signature: ……. Official Stamp: …….
Final Year Project Coordinator	Name: Nadiah Arsat Signature: ……. Official Stamp: …….
Department Dean	Name: Assoc. Prof. Dr. Basheer Riskhan Signature: ……. Official Stamp: …….

ACKNOWLEDGEMENT

We would like to extend our sincere gratitude to everyone who contributed to the successful completion of this project. First and foremost, we're deeply thankful to **Professor Dr. Khalid Hussain**, our supervisor, for his invaluable guidance, insightful feedback, and constant support. His expertise and advice have been essential in helping us navigate the complexities of web application security and vulnerability assessment.

We also wish to express our heartfelt appreciation to our group members, **Elzobeir Salih Mahmoud Mohamad** , and **Afrah Hamed Ali Noor Mohammed** , for their collaboration, dedication, and teamwork. Each of us played a crucial role in the research, development, and testing of the Machine Learning based intrusion detection system , and it was through our collective efforts that we were able to bring the project to completion.

Last but not least, we would like to thank the **School of Computing and Informatics** at **Albukhary International University** for providing us with the resources and tools necessary to carry out this project. The technical support, academic resources, and conducive learning environment greatly contributed to our ability to tackle this challenging task.

TABLE OF CONTENTS

DECLARATION..... Ⅰ
APPROVAL..... Ⅱ
ACKNOWLEDGEMENT..... Ⅲ
TABLE OF CONTENTS..... Ⅳ
LIST OF ABBREVIATION..... Ⅴ
LIST OF FIGURES..... Ⅵ
ABASTRACT..... Ⅶ
CHAPTER ONE..... 1
INTRODUCTION..... 1
1.1 Overview..... 1
1.2 Background..... 1
1.3 Problem Statement..... 2
1.4 Research Questions:..... 2
1.5 Research Objectives:..... 2
1.6 Project Scope..... 2
CHAPTER TWO..... 4
LITERATURE REVIEW..... 4
2.1 Literature Review..... 4
2.2 Literature Review Table:..... 8
2.3 Critical Findings from the Literature Review..... 15
2.4 Summary of Literature Review:..... 15
CHAPTER THREE..... 16
METHODOLOGY..... 16
3.1 Overview of the Methodology..... 16
3.2 Requirements Gathering..... 17
3.3 Requirement Analysis..... 17
3.4 System Design..... 18
3.5 Capture Traffic System Development..... 18
3.6 Machine Learning Implementation..... 19
3.7 Alert System Implementation..... 19
3.8 Testing..... 19
CHAPTER FOUR..... 23
IMPLEMENTATION..... 23
4.1 Capture traffic:..... 23
4.2 Machine Learning System..... 26
4.3 Alert System:..... 33
4.4 System Deployment and Integration..... 36
CHAPTER FIVE..... 39
RESULT AND DISCUSSION..... 39
5.1 Real-Time Intrusion Detection..... 39
5.2 Machine Learning Model Evaluation..... 39

5.3 Email Alert System.....41
CHAPTER SIX..... 43
CONCLUSION.....43
6.1 Contribution to Social Business..... 43
6.2 Limitations and Future Work.....43
6.2.1 Limitations..... 43
6.2.2 Future Work..... 44
6.3 Conclusion.....44
References..... 46
Appendices.....48

LIST OF ABBREVIATION

AI	Artificial Intelligence
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
DoS	Denial of Service
DDoS	Distributed Denial of Service
FPR	False Positive Rate
FNR	False Negative Rate
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
ML	Machine Learning
NSL-KDD	Network Security Laboratory - Knowledge Discovery and Data Mining
RF	Random Forest
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol
SVM	Support Vector Machine
UNSW-NB15	University of New South Wales Network-Based 15 Dataset
XGBoost	eXtreme Gradient Boosting

LIST OF FIGURES

Figure 1: System development methodology
Figure 2: System architecture
Figure 3: Use case diagram
Figure 4: Activity diagram
Figure 5: Traffic Capture workflow
Figure 6: Source code for interface selection
Figure 7: Network interface displayed on screen when running the system
Figure 8: Code for targeted protocols
Figure 9: Code for targeted services
Figure 10: ML workflow
Figure 11: Import of libraries
Figure 12: Uploaded dataset
Figure 13: Dataset distribution
Figure 14: Feature selection
Figure 15: Balanced dataset
Figure 16: Encoding categorical features
Figure 17: Features normalization
Figure 18: Data after balance
Figure 19: Models training
Figure 20: Feature distribution
Figure 21: Alert system diagram
Figure 22: Configuration and initialization of the server for email alert
Figure 23: Format of the email that will be sent
Figure 24: System workflow
Figure 25: System detecting traffic
Figure 26: Comparison of the trained algorithms
Figure 27: Confusion matrix of best model
Figure 28: Sample email
Figure 29: Gantt Chart
Figure 30: Plotting boxplots for numerical features
Figure 31: Confusion matrix for Random Forest
Figure 32: Confusion matrix for Decision Tree

ABASTRACT

Intrusion Detection Systems basically are cybersecurity devices that monitor network traffic to identify intrusion attempts and alert users to malicious activities. The three major methods of IDS include signature-based, anomaly-based, and hybrid; each one of them has advantages and disadvantages. Signature-based methods provide high accuracy for known threats but require constant updating and hence are susceptible to new and emerging attacks. On the other hand, anomaly-based methods do an excellent job of finding unknown threats by identifying deviations from normal patterns; however, they usually operate at lower accuracy with high false alarm rates. Hybrid approaches, which combine the strengths of both, provide full detection capabilities but come at higher computational costs. This paper reviews IDS functionality improvement using Machine Learning techniques that consider datasets, algorithms, and evaluation metrics. Algorithms such as Naïve Bayes, Support Vector Machines, and K-Nearest Neighbors find a large number of applications in anomaly-based intrusion detection systems to deal with large volumes of data and cope with the ever-evolving cyber world. However, the effectiveness of these models relies heavily on the structure and applicability of the datasets used. The research adopts a two-stage experimental approach, leveraging the NSL-KDD and UNSW-NB15 datasets. In the first stage, various ML algorithms are evaluated based on performance metrics such as accuracy, True Positive Rate (TPR), and False Positive Rate (FPR). The second step involves the enhancement of the model by selecting the most effective algorithm and optimizing its performances. One such finding from the study is that all outdated datasets such as KDDCup99 and its variant NSL-KDD are insufficient to represent complex cyber threats/behaviors in modern times, thus narrowing the scope for IDS research. This research further supports the importance of soft computing methods for enhancing known intrusion classification issues, while also pointing out various challenges that will persist in attempting to detect new or modified attacks. A resulting IDS developed on advanced ML algorithms is remarkable in gains both for real-time detection and adaptability-cost efficiency to offer robust protections for SMEs and essential sectors such as healthcare and non-profits.

CHAPTER ONE

INTRODUCTION

1.1 Overview

Increased complexity and frequency of cyber threats make it difficult for classic IDS that use signature or rule-based methods to effectively respond to emerging threats. This project seeks to establish a more advanced IDS, which has incorporated machine learning approaches in the identification and categorisation of network intrusions. Specifically, the proposed system aims to enhance the real-time detection and classification performance using various types of the UNSW-NB15 dataset, including DoS, Exploit, Fuzzers, Analysis, and Backdoor attacks. Important facets of the project include data pre-processing for dealing with imbalance datasets, selection and training of multiple machine learning models, and model evaluation that employ accurate assessment criteria that may involve accuracy, precision, recall, and F1-score. The final objective is to establish a financially feasible IDS model that can effectively mitigate contemporary threats, and positively impact SME organisations.

1.2 Background

In today's digital landscape, the Internet has become one of the essential aspects that opens opportunities in the present-day society as well as poses numerous threats in the form of cybersecurity issues. It is vital to protect these domains as the use of web-based applications increases each year. Conventional IDS have been implemented to safeguard network infrastructure from threats through detection and prevention; they however rely on signature based or rule-based approaches, which are not effective in the current complex threat landscape. Traditional IDS fail in real-time identification and categorization of new or morphing threats that are not recognizable through attack patterns. This is further compounded by under-represented class attack types that lead to classifications, hence hampering the effectiveness of the detection rate. To address these challenges, this study suggests the enhancement of an advanced IDS that will incorporate features of the machine learning algorithms. Due to its high variability of attack types and normal traffic, the chosen datasets UNSW-NB15 allows discovering the machine 7 learning algorithms' efficiency, as well as accurately solving the problem of imbalanced data and real-time traffic analysis. In this context, this approach is proposed as an improved solution for IDS, better tailored to modern threats and extending its applicability to a broader range of organisational environments.

1.3 Problem Statement

Traditional Intrusion Detection Systems (IDS) often struggle to detect and respond to various cyber-attacks in real-time, leading to significant breaches and damage. To be precise, these systems are usually based on the signature-based and rule-based methods, which are not suitable in real time environments in regard to the emerging threats. Also, datasets used in IDS research including NSL-KDD offer limited simulation of present-day network environments that often results in poor performance.Constant expansion of the internet use and reliance on network-dependent platforms have boosted the demand for better IDS solutions. Malware attacks, data breaches, and denial-of-service attacks persist, and new strains of attacks cause considerable damage and financial losses while threatening valuable information. To meet these challenges ML techniques have come out as quite useful attempts. These methods help in improving the rates of identification of both 'novel' and 'known' threats more easily by use of anomaly-based detection. Still, existing ML and DL-based IDS frameworks are faced with the following issues: the framework's scalability, high false positive rates, and the use of outdated datasets. This research seeks to address these problems by enriching IDS frameworks, minimizing false positives, as well as increasing the capability to make timely changes in response to new threats .

1.4 Research Questions:

- What are the key limitations of traditional Intrusion Detection Systems in detecting advanced cyber-attacks, and how can modern machine learning approaches address these challenges?
- How can machine learning algorithms be optimised to enhance the real-time detection capabilities of Intrusion Detection Systems against sophisticated and complex cyber-attacks?
- What techniques can be employed to improve the accuracy of attack classification in Intrusion Detection Systems, particularly when dealing with imbalanced datasets?

1.5 Research Objectives:

- Develop an Intrusion Detection System (IDS) leveraging machine learning to detect and classify network attacks in real time.
- Trigger real-time email alerts upon detection of malicious activities, ensuring prompt notification and response to potential threats.

1.6 Project Scope

- Enhance the effectiveness of Intrusion Detection Systems (IDS) by addressing their limitations using machine learning techniques.
- Develop a real-time detection system capable of accurately identifying complex cyberattacks for swift and reliable threat responses.
- Create faster and more precise classification methods for network intrusions, focusing on handling imbalanced datasets to improve detection accuracy.
- Use machine learning (ML) algorithms to overcome the weaknesses of traditional signature- and rule-based methods.

- Increase detection rates while reducing false positive rates, ensuring a more dependable and efficient IDS.

CHAPTER TWO

LITERATURE REVIEW

2.1 Literature Review

In a survey conducted by Zhang, Wang, and Lee (2022) on contemporary IDS with machine learning, authors found out that CNN and RNN are the most used techniques for analyzing network traffic. These models were shown to achieve very high accuracy in identifying APTs and anomalies that could be used for real-time detection in complex evolving networks based on the work done by Zhang et al., 2022.

Sanchez and Li (2022) looked at the important problem of analyzing IDS data with an unbalanced dataset. They suggested that fake attack data can be generated through the state of the art method called Generative Adversarial Networks (GANs) for balancing the datasets and to increase the detection rates. This method yields better performance on attack types that are rare in occurrence since the pertinence of sufficient data is maneuvered (Sanchez & Li, 2022).

Kumar & Das (2022) further explained how to use unsupervised learning for Zero-day attack detection. A clustering-based system that integrated anomaly detection models was studied by them and the findings indicated that through new clustering approaches the baseline detectors can identify new forms of attacks without prior exposure. This approach proves to be more useful in detecting zero-day threats, which are not detected by other systems based on signatures (Kumar & Das, 2022).

In examining real-time IDS trends, Nguyen & Pham (2022) explained that machine learning models which integrated both decision trees and deep learning were more effective. Hid their research that improved real-time detection efficiency reaching almost completely excluding false alarms with indication on the need in unification of machine learning approaches to enhance IDS's work (Nguyen & Pham, 2022).

Drawing from the literature, Miller and Roberts (2023) examined the effect of ML in IDS with XAI (Explainable Artificial Intelligence) as a newly emerging area of focus. There's an argument that by improving the interpretability of machine learning models, security teams can increase their understanding of how specific threats are identified and more effectively respond to them in time (Miller & Roberts, 2023).

More real-world applications and growth in IDS of machine learning are evident by the progressive research efforts made recently to acknowledge and solve problems such as modeling imbalance data, real-time detection and zero-day threats. Next research directions are expected to be oriented on combining the approaches and improving the explainability in intelligent IDS solutions (Kevric et al., 2017).

Kevric et al. (2017] for instance examined the classifiers made of single tree algorithms like the decision trees, random forest, and extra trees for network intrusion detection. They applied the NSL-KDD dataset where the work was illustrated to achieve higher accuracy, precision, recall, and F1-score than using single classification algorithms on combined classifiers (Kevric et al., 2017).

Lee and Washizaki (2017) assessed the machine learning based IDS by using KNN, SVM and decision tree algorithms on the NSL-KDD set of benchmarks. According to their results, decision tree algorithms have higher accuracy in identifying various types of attacks.

This was presented in Taher et al. (2019) where the authors developed IDS by employing supervised SM techniques with feature selection. They tested three types of algorithms namely logistic regression, random forest, gradient boosting algorithms using NSL-KDD dataset. In terms of accuracy, precision, recall and F1-score random forest was found to be better than others (Taher et al., 2019).

The NSL-KDD dataset is more feasible in IDS research than the original KDD Cup 1999 dataset for its enhanced non-redundancy feature in attack coverage. It helps to establish a standard for the assessment of the efficiency of IDS based on ML (Taher et al., 2019).

Al-Mimi et al. proposed an applied IDS improvement business to defend HTTP services from attack employing classifiers from machine Study. They used the UNSW-NB15 dataset and put forward the ML-HTTP model. Also, from several algorithms that were experimented with, Random forest and Logistic regression marked the highest accuracy of 96.32%Bearing in mind the importance of machine learning in detecting HTTP attacks (Al-Mimi et al., 2023).

Somashekar and Boraiah used the prediction-level fusion of the Gradient Boosted Tree and Random Forest classifier for network intrusion detection. In experiments on the NSL-KDD dataset, they achieved a classification accuracy of 96.31%, and showed that hybrid models and retraining help to improve the work of IDS against unknown types of attacks (Somashekar & Boraiah, 2023).

In their work, the papers of Jaradat and his colleagues also used the CICIDS2017 data set to build a machine learning based IDS. It looked specifically at feature extraction and data preprocessing, though quite good accuracies that ranged between 96% and a maximum of 98.6% were obtained from classifiers such as SVM and Decision Trees. The study also pointed out how ML can enhance the performance of detection rates and also minimize the false positive cases in cybersecurity applications (Jaradat et al., 2022).

It is a systematic review of Machine Learning techniques for IDS based on IoT conducted by (Idris et al). They assessed standard datasets like NSL-KDD and BoT-IoT and realized ensembled approaches with Logistic Regression and Neural Network as promising. The analysis highlighted a need to provide lightweight IDS solutions to IoT ecosystems (Idris et al., 2023).

Hence, Note and Ali compared machine learning and deep learning algorithms for IDS using datasets such as NSL-KDD and UNSW-NB15. The results show that Random Forest was the best performing model, however CNN and RNN with high accuracy rate presented a computational issue. It validated the promise of higher models in cybersecurity of which their study comprised (Note & Ali, 2022).

Sadeghian et al. also used the NSL-KDD dataset to compare three ML techniques for IDS with eight algorithms. This was an encouraging discovery to them because deep learning approaches were found to be the most efficient compared to other methods such as Naïve Bayes. This research highlighted a strong need for effective methods necessary in identifying outliers or anomalies quickly (Jani et al., 2018)

Nachiket et al used feature classification and outlier detection for proper identification of IDS for their enhancement. They attempted to use a variety of datasets such as the NSL-KDD and the KDDCup99 datasets and to classify them using classifiers like Random Forest and Support Vector Machines. Sainis et al. (2018) suggested that feature selection is capable of minimizing noise and maximizing the detection rate, as confirmed by their study

Al-Hadhrami et al. introduced an anomaly-based IDS with support of different supervised machine learning techniques. They also obtained good results considering the detection rates for DDoS attacks such as Random Forest and Gradient Boosted Trees. They concluded their work by supporting the factors they identified during feature selection, and ensemble learning as key to optimum IDS performance (Al-Hadhrami et al., 2022).

Widodo and Aji introduced IDS and summed up papers regarding intrusion detection systems, and machine learning techniques in particular. They compared such models as Random Forest and Gradient Boosting and underlined their advantages connected with higher accuracy and higher speed of calculations. The study called for a realistic dataset to manage the emerging cybersecurity threats (Widodo & Aji, 2021).

Ahmad et al. reviewed the use of ML for intrusion detection in networks using the NSL-KDD data set. They demonstrated that the use of ensemble methods is more effective than the application of one or several classifiers, with accuracy of the results being rather high. This research highlighted the significance of ensemble learning when it comes to identifying the complex threats (Ahmad et al., 2023)

Plaka et al. presented a Deep learning framework for IDS with more emphasis on time based attacks such as DDoS. LSTM models are very efficient when it comes to identifying sequential patterns of network traffic, as their authors demonstrated and compared to traditional classifiers in terms of precision and recall (Plaka et al., 2021).

Kevric and his team took advantage of Decision Trees and Neural Networks in setting up a combined classifier. In the NSL-KDD and KDDCup99 dataset, their work achieved higher detection rates for difficult to detect threats such as R2L and U2R. They also focused on the concept of combining different methods to increase accuracy in IDS (Kevric et al., 2017)

Taher et al. therefore used feature selection methodology together with supervised ML classifiers to enhance IDS effectiveness. This was in line with much improved accuracy of Decision Trees and Naive Bayes especially where feature selection for dimension reduction was applied (Taher et al., 2019)

The work of Munir et al. was to analyze the use of ontology-based IDS methods for protecting HTTP services. Their semantic analysis enhanced accurate identification of novel attacks such as HTTP smuggling and splitting, which underlines the necessity of domain knowledge in IDS design (Munir et al., 2022).

The authors Başkaya and Samet used the anomaly-based methods to identify the DDoS attack. They proved that feature engineering using ML classifiers including MLP, produce response rates more than 99 percent of questions and statements aiming at recognition and limitation of the imbalance of classes in the data set.

Hong et al presented an SDN assisted solution for dealing with slow HTTP DDoS attacks. In their SHDA model, HTTP requests to the system were effectively early terminated below 2048 bytes, thereby preserving system consistency and minimizing requant computing when an attack was imminent (Hong et al., 2023).

Marir et al. have put forward a deep belief network integrated with ensemble SVM for detecting anomalies in the distributed networks. A similar approach yielded high detection levels of anomalies and low false alarms where the real-time systems are applied successfully (Marir et al., 2018).

Ayeni et al investigated CNN based models in modern network environments for intrusion detection . They discovered that random imbalances in datasets can be mitigated while convolution layers are useful in developing high dimensional features (Ayeni, Dorado, Ponce, Wang, & Ji, 2023).

2.2 Literature Review Table:

TITLE	YEAR	DATA SET	OBJECTIVES	ALGORITHMS	Authors
Survey of current intrusion detection systems (IDS) with machine learning	2022	Various network traffic datasets	Identify effective machine learning models for IDS	CNN, RNN	Zhang, Y., Wang, L., & Lee, S.

Handling unbalanced datasets in IDS using Generative Adversarial Networks	2022	Unbalanced IDS datasets	Enhance dataset balance for rare attack types	GANs	Sanchez, R., & Li, X.
Application of unsupervised learning for zero-day attack detection	2022	Clustering-based IDS datasets	Detect zero-day attacks without prior training	Clustering techniques	Nachiket, S., Srivastava, D., & Singh, R.
Real-time IDS trends emphasizing hybrid models	2022	Real-time IDS datasets	Improve real-time detection efficiency	Decision Trees, Deep Learning	Nguyen, T. H., & Pham, Q. M.
Impact of machine learning on IDS with Explainable AI	2023	Various IDS datasets	Enhance explainability of machine learning models	Explainable AI models	Miller, J., & Roberts, T.
Classifiers combining multiple tree algorithms for IDS	2017	NSL-KDD	Boost accuracy with combined classifiers	Decision Trees, Random Forest, Extra Trees	Kevric, J., Jukic, S., & Subasi, A.
Machine learning-based IDS evaluation on NSL-KDD dataset	2017	NSL-KDD	Evaluate machine learning algorithms for IDS	KNN, SVM, Decision Trees	Dutta, M., Krishna, C. R., Kumar, R., & Kalra, M.

IDS using supervised machine learning techniques with feature selection	2019	NSL-KDD	Optimize IDS performance with feature selection	Logistic Regression, Random Forest, Gradient Boosting	Taher, K. A., Jisan, B. M. Y., & Rahman, M. M.
Enhanced IDS for HTTP attacks using machine learning classifiers	2023	UNSW-NB15	Protect HTTP services using ML classifiers	Random Forest, Logistic Regression	Al-Mimi, H., Hamad, N. A., Abualhaj, M. M., et al.
Prediction-level fusion model for network intrusion detection	2023	NSL-KDD	Combine classifiers for improved performance	Gradient Boosted Trees, Random Forest	Note, J., & Ali, M.
Machine learning approach for IDS using the CICIDS2017 dataset	2022	CICIDS2017	Improve detection rates and reduce false positives	SVM, Decision Trees	Jaradat, A. S., Barhoush, M. M., & Bani Easa, R. S.
Systematic review of machine learning techniques for IoT-based IDS	2023	NSL-KDD, BoT-IoT	Evaluate lightweight IDS solutions for IoT	Hybrid models: Logistic Regression + Neural Networks	Idris, M., Yasin, S. M., & Audi, G. H.

Comparative analysis of machine learning and deep learning algorithms for IDS	2022	NSL-KDD, UNSW-NB15	Compare ML and DL models for IDS	Random Forest, CNN, RNN	Jani, K., Lalwani, P., Upadhyay, D., & Potdar, M. B.
Evaluation of machine learning algorithms for IDS	2018	NSL-KDD	Evaluate ML algorithms for anomaly detection	Deep Learning, Naïve Bayes	Khan, S., Sivaraman, E., & Honnavalli, P. B.
Feature classification and outlier detection for IDS	2018	NSL-KDD, KDDCup99	Improve accuracy using feature selection	Random Forest, SVM	Nachiket, S., Srivastava, D., & Singh, R.
Anomaly-based IDS for detecting DDoS attacks	2022	Various DDoS datasets	Detect DDoS attacks using ensemble learning	Random Forest, Gradient Boosted Trees	Baskaya, D., & Samet, R.
Review of intrusion detection systems focusing on machine learning methods	2021	Various IDS datasets	Review hybrid IDS models for cybersecurity	Random Forest, Gradient Boosting	Kumar, A., & Das, M.
Deep learning framework for IDS focusing on	2021	Time-based attack datasets	Develop framework for detecting time-based attacks	LSTM	Plaka, M.

time-based attacks					
Combined classifier approach using Decision Trees and Neural Networks	2017	NSL-KDD, KDDCup 99	Improve detection rates with combined classifiers	Decision Trees, Neural Networks	Kevric, J., Jukic, S., & Subasi, A.
Featureselection techniques with supervised ML classifiers	2019	NSL-KDD	Optimize performance using feature selection	Decision Trees, Naïve Bayes	Taher, K. A., Jisan, B. M. Y., & Rahman, M. M.
Ontology-based IDS methods for securing HTTP services	2022	HTTP-specific datasets	Secure HTTP services with semantic approaches	Domain-specific ML approaches	Widodo, T., & Aji, A. S.
Anomaly-based methods for detecting DDoS attacks	2023	DDoS-specific datasets	Enhance detection of DDoS attacks with feature engineering	MLP	Al-Hadhrami, M., Al-Qurishi, M., & Yasin, S.
SDN-assisted framework for mitigating slow HTTP DDoS attacks	2023	HTTP DDoS datasets	Mitigate slow HTTP DDoS attacks with SHDA	SHDA model	Hong, K., & Choi, J.

Deep belief network combined with ensemble SVM for detecting anomalies	2018	Distributed network datasets	Reduce false positives with ensemble SVM	Deep Belief Network, SVM	Marir, N., Wang, H., Feng, G., Li, B., & Jia, M.
CNN-based models for intrusion detection	2023	Modern network datasets	Address dataset imbalances for intrusion detection	CNN	Ayeni, O. A., Ewa, S. C., & Owolafe, O.

2.3 Critical Findings from the Literature Review

The literature review emphasizes important progress in the use of machine techniques for intrusion detection systems (IDS). Main observations highlight the efficiency of hybrid models that integrate machine learning algorithms such as Random Forest and Gradient Enhancing.

These hybrid techniques enhance precision while simultaneously lowering false alarm rates, tackling one of the key issues in IDS.

Utilizing methods like clustering to identify zero-day attacks, demonstrating the flexibility of IDS to advancing cyber dangers.

The evaluation additionally highlights the significance of tackling issues like imbalanced datasets and limitations on resources, especially in IoT settings. The implementation of feature selection and engineering is consistently emphasized as essential for enhancing IDS effectiveness and minimizing false positives.

Additionally, the (NSL-KDD)dataset becomes a commonly utilized standard for assessing IDS.

Effectiveness because of its thorough depiction of attack situations, offering a trustworthy basis for comparative examination.

2.4 Summary of Literature Review:

The literature shows that machine learning has become essential in contemporary IDS, providing strong solutions for identifying a variety of complex cyber threats. Hybrid models and ensemble techniques often surpass conventional methods, improving detection precision and system dependability. The implementation of unsupervised learning methods and XAI demonstrates the increasing demand for flexibility and clarity in IDS, enhancing their efficiency in dynamic and real-time network settings. The significance of preprocessing, including feature selection and dataset balancing, is crucial in enhancing IDS performance and tackling prevalent issues like false positives and dataset imbalance.

In general, the research highlights a significant emphasis on utilizing sophisticated algorithms to address emerging cyber threats, while also recognizing practical limitations like computational resources and real-time detection. The results highlight the significance of innovation in creating IDS systems that are efficient and scalable, especially in resource-limited contexts such as IoT and cloud environments. Through the integration of these innovations, IDS are more effectively equipped to protect contemporary networks from progressively intricate cyber threats.

CHAPTER THREE

METHODOLOGY

3.1 Overview of the Methodology

The methodology details the procedures and techniques employed in the design, validation, and implementation of the Real-Time Intrusion Detection System (IDS). The IDS provides continuous surveillance for the network traffic and analyzes it in real time in order to classify and identify possible threats by determining network traffic patterns consistent with certain types of malicious behavior. The system uses ML approaches which include (Random Forest, Decision Tree and XGBoost), to determine whether the traffic is benign or malicious.

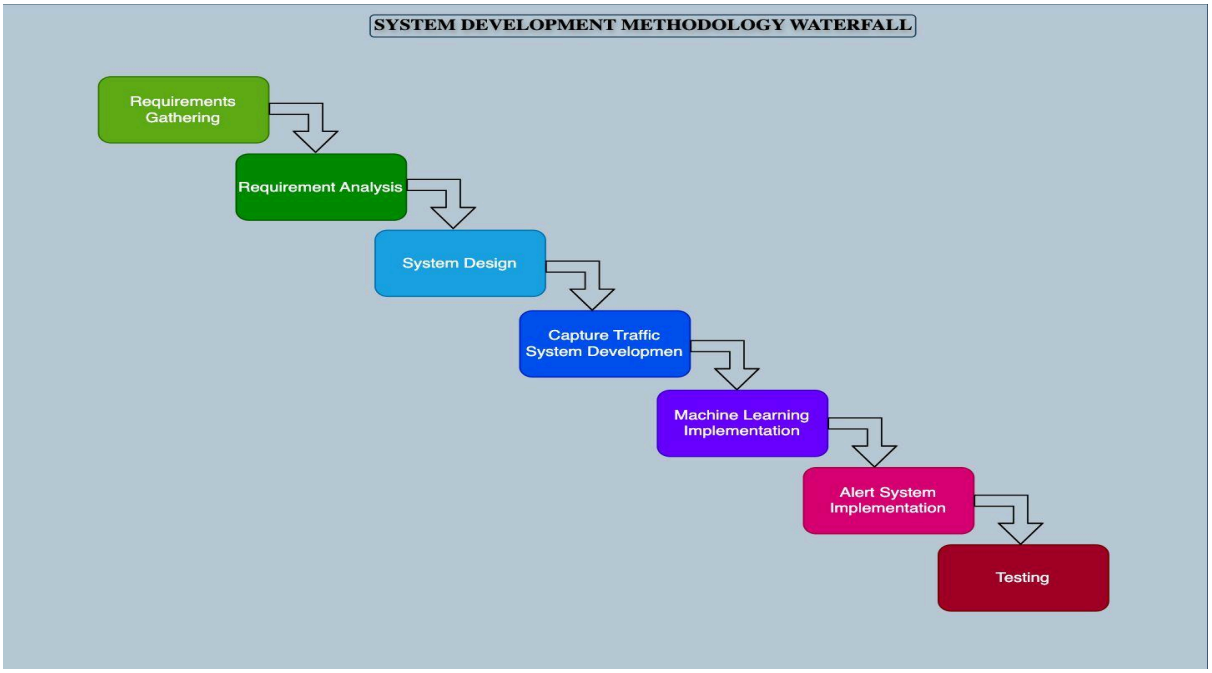


Figure 1: system development methodology

3.2 Requirements Gathering

The first step in creating the system was determining all the requirements needed to be incorporated. This included determining the main features of the intrusion detection system and its operational capabilities. In more tangible terms, looking into the question of what kind of network threats the system should be able to detect, this was extensively researched with relevant sources to figure out what tools and data sources would be required. This step was fundamental in making sure the system would perform as expected.

3.3 Requirement Analysis

This engineering phase concerned the decomposition of the requirements that were collated into clear actionable tasks, this was deemed necessary for the success of the project in order to achieve its objectives. The primary aim of this task was to identify the technical and functional requirements of the intrusion detection system. scrutinized the dataset (UNSW-NB15) in terms of its sufficiency in features to facilitate the training of the machine learning models. The protocol type, packet statistics, data transfer rates, and attack types were some of the salient features considered in detecting malicious traffic. Similarly, analyzed the system tools a necessity, Scapy for packet sniffing, Python to libraries for data preprocessing, and machine learning algorithms such as Random Forest and XGBoost for classification.

3.4 System Design

In this phase, we worked on the design of the modular and scalable architecture for the intrusion detection system. This included identifying the key components of the system that were traffic monitoring, feature extraction, data preprocessing, classification of traffic using some form of machine learning, and an alert mechanism. Each component was required to function independently and yet be able to integrate with the rest of them. For traffic monitoring, Scapy was picked for the online sniffing of network packets as well as extracting crucial components such as protocol, packet size and the transfer of data rate. The machine learning module was to use trained classifiers such as Random Forest, Decision Tree and XGBoost to determine whether the identified traffic was normal or malicious. Also designed were data preprocessing modules like feature scaling, encoding and alignment which are used both in preparation of offline datasets and real-time traffic processing. To facilitate access to email anytime malicious activity was detected a system was implemented that sent emails with relevant information such as type of attack and the time it happened. Such a structure designed the system's efficiency, flexibility, as well as the ability to work in real time environments without compromising detection accuracy.

3.5 Capture Traffic System Development

The phase of the development of the traffic capture system was focused on developing a module for the monitoring and analyzing of real time network traffic. With the use of Scapy, Iequipped a packet sniffing module that had the capability of capturing and parsing packets that had been received from a particular network interface. The system was programmed to be able to retrieve critical features such as protocol name, service type, packets sent per form and data transfer sizes also referred to as bytes. added a protocol identification and a protocol inference mechanism that determines the type of service offered by the port numbers for instance HTTP, BitTorrent among others. These extracted features were then constructed in a well-defined format that would make it easier for the machine learning model to utilize. The

module was also designed to cater for some breaches like the integrity of the data ensuring required measures were set to tackle all cases such as non-receipt of packets or incompleteness of packets. By automating this process, the system was capable of monitoring traffic continuously in real time, classifying data with ease and linking with the rest of the machine learning pipeline. This phase was critical for assuring of the system's ability to manage real traffic operations and to identify possible threats without delay.

3.6 Machine Learning Implementation

The deployment of machine learning was the most crucial phase in developing a real-time intrusion detection system with the capability of determining whether network traffic is normal or malicious. Preprocessing of the UNSW-NB15 dataset involved handling missing values, encoding categorical features, and normalizing numerical features. The imbalance in classes or major classes of normal and malicious traffic were dealt with by SMOTE oversampling an underrepresented class. This was done so that all forms of attacks by the machine learning model would be duly detected. Random Forest, Decision Tree, and XGBoost were some machine learning types trained, and their performance measured using metrics such as accuracy, precision, recall, and F1 score. Compressed results were used to identify the most superior model for deployment. A model was saved in (joblib), after which integration into the real-world system would be completed. Thus, this created a robust, reliable, and optimized machine learning throughput or pipeline that allows real-time classification of network traffic.

3.7 Alert System Implementation

The primary objective of the alert system implementation was to ensure that the intrusion detection system could react to intrusions in a timely manner by directly informing the user. This procedure comprised the development of a mechanism for sending email alerts whenever the machine learning model recognized an attack. Through the SMTP protocol, put the unit up to use detailed notifications consisting of the attack's nature, the time when it happened, the protocol used, the type of service, and packet statistics. The alert system functions seamlessly with the real-time monitoring and classification modules, thus ensuring that notifications are sent out immediately a threat is detected. In order to make the email messages more user-friendly, choose the way they look which is structuring them in such a way that they give a short but thorough account of the intrusion, the alert information, and the actions that are demanded of the user. This step was instrumental in filling out the seemingly wide gap between the detection of an intrusion and the remediation of the system, so it can be a more efficient tool for practical use in real-world environments.

3.8 Testing

To test the system, use two virtual machines running on VMware that are both Kali Linux(an OS that is used by cyber security experts to attack or defend systems) . The setup has one machine as the Blue Team (defender) and the other one as the Red Team (attacker). The Red

Team performs various types of topics about malicious traffic on the internet, which the Blue Team then uses to network and check the developed intrusive system. the system Transited by using real-world attack scenarios to test the system's capability of identifying and responding to threats. This method of testing made the system to be efficient under practical conditions and showed its reliability in identifying intrusions that occur in real-time.

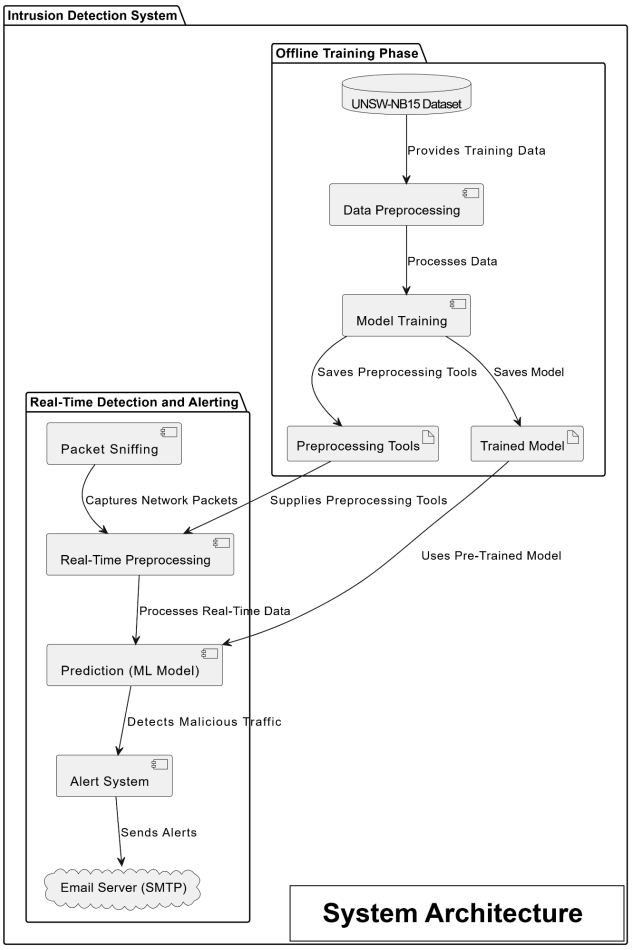


Figure 2: system architecture

The system architecture consists of two primary phases: the offline Training Phase and the Real-Time Detection and Alerting Phase. During the Offline Training Phase, the UNSW-NB15 dataset undergoes processing through a data preprocessing module, where features are cleaned, encoded, and scaled. This processed data is then utilized to train the machine learning model in the model training step, resulting in two outputs: the trained model and the preprocessing tools (such as encoders and scalers), which are stored for use in real-time detection. In the Real-Time Detection and Alerting Phase, live network traffic is captured using a packet-sniffing module. The captured traffic is then processed through a real-time preprocessing module, which applies the same transformations as in the training phase using the saved preprocessing tools. The preprocessed data is subsequently input into

the trained machine learning model, which classifies the traffic as either normal or malicious. If malicious traffic is identified, the alert system is activated to send detailed notifications via an email server (SMTP), ensuring timely alerts for potential threats. This design facilitates a seamless transition from offline training to real-time detection and alerting.

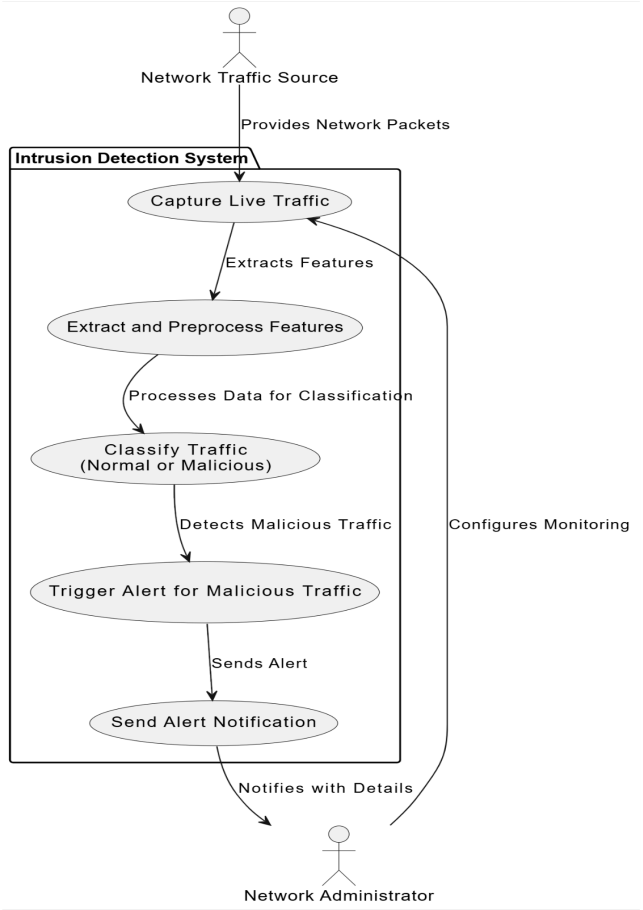


Figure 3: use case diagram

The use case diagram shows how two actors, the Network Traffic Source and the Network Administrator, interact with the Intrusion Detection System. The process starts with the system capturing live traffic from the network source, where packets are analyzed. Itthenextracts and preprocesses important features such as protocol, packet size, and rate. The preprocessed data is classified using a machine learning model to identify whether the traffic is normal or malicious. If the system detects malicious traffic, it triggers an alert, sending a detailed notification to the Network Administrator that includes information about the type of attack and other relevant details for swift action. This flow ensures effective monitoring and timely alerts for potential security threats.

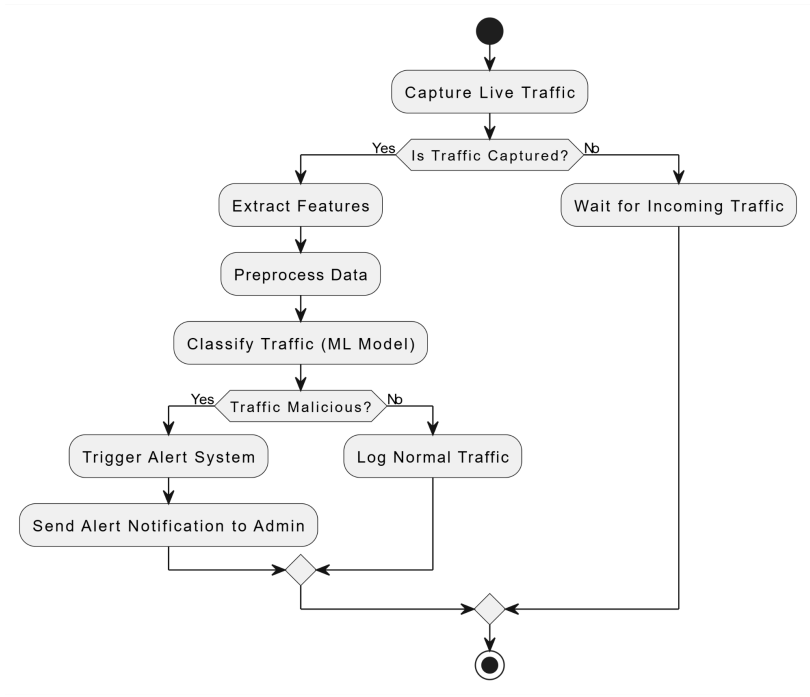


Figure 4: activity diagram

The activity diagram outlines the process of the intrusion detection system. It starts by capturing live network traffic. If traffic is successfully captured, features are extracted, and the data is preprocessed before being classified by the machine learning model. If the traffic is identified as malicious, the alert system is triggered, and a notification is sent to the administrator. If the traffic is normal, it is logged for future reference. If no traffic is captured, the system waits for incoming traffic. The process ensures efficient monitoring and timely responses to potential threats.

CHAPTER FOUR

IMPLEMENTATION

4.1 Capture traffic:

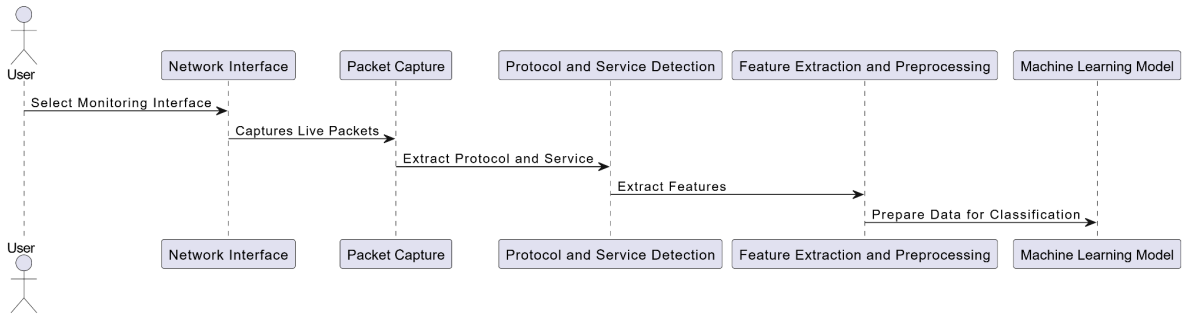


Figure 5: shows Traffic Capture workflow

1. Traffic Capture Diagram

Explanation: The traffic capture process is essential for real-time intrusion detection. Here's a detailed step-by-step explanation of the diagram:

1. User Interaction:
    - The process begins with the User selecting the network interface to monitor. A list of available interfaces is displayed, allowing the user to choose the one relevant to the target traffic.
2. Network Interface:
    - The selected Network Interface acts as the entry point for capturing network packets. It continuously listens to incoming and outgoing traffic on the specified network interface.
3. Packet Capture:
    - The Packet Capture module, powered by tools like scapy, captures live packets in real-time. This raw traffic data includes various protocols (e.g., TCP, UDP, ICMP) and payloads.
4. Protocol and Service Detection:
    - The captured packets are analyzed to detect their protocol (e.g., TCP, UDP) and associated service (e.g., HTTP, HTTPS, DNS). This detection is critical for understanding the type of communication and narrowing down suspicious activities.
5. Feature Extraction and Preprocessing:
    - The relevant features are extracted from the packets, such as:
        - Number of packets sent (spkts) and received (dpkts)
        - Data size sent (sbytes) and received (dbytes)
        - Data rate, duration, traffic direction, and size variance
    - The features are preprocessed through one-hot encoding for categorical data (protocol and service) and normalization for numerical data. This ensures compatibility with the machine learning model for classification.
6. Output:
    - The Feature Extraction and Preprocessing module prepares the extracted features into a structured format aligned with the trained machine learning

model's input requirements. This step bridges raw network traffic with predictive analysis.

1. Selecting Network Interface

```
from scapy.all import sniff, get_if_list

# List available interfaces
interfaces = get_if_list()
print("Available network interfaces:")
for idx, iface in enumerate(interfaces):
    print(f"{idx + 1}. {iface}")
selected_idx = int(input("Select the interface number to monitor: ")) - 1
selected_iface = interfaces[selected_idx]
print(f"Monitoring interface: {selected_iface}")
```

Figure 6: source code for interface selection

Explanation:

- get_if_list(): Retrieves a list of all available network interfaces on the system.
- User Interaction: Displays the interfaces, prompting the user to select one by entering its corresponding number.
- Selection: The selected interface is stored in selected_iface, which will be used for real-time packet capture.
- Purpose: This step is critical for defining the scope of monitoring, ensuring the system listens to traffic from the desired network.

```
└─# python real_time_ids.py
Available network interfaces:
1. lo
2. eth0
Select the interface number to monitor: 2
Monitoring interface: eth0
Starting packet capture ...
```

Figure 7: network interface displayed in screen when run the system

2. Packet Sniffing and Protocol Detection

```python
# Start sniffing
def process_packet(packet):
    if packet.haslayer('IP'):  # Ensure the packet contains an IP layer
        # Protocol detection
        if packet.haslayer('TCP'):
            proto = 'tcp'
        elif packet.haslayer('UDP'):
            proto = 'udp'
        elif packet.haslayer('ICMP'):
            proto = 'icmp'
        else:
            proto = 'unknown'
```

Figure 8: code for targeted protocols

Explanation:

- sniff(): Captures live packets from the selected network interface.
- Protocol Detection:
    - The system checks the packet for specific layers (e.g., TCP, UDP, ICMP) to identify the communication protocol.
    - If no matching layer is found, the protocol is marked as unknown.
- IP Layer Check: Ensures the packet contains an IP layer, which is necessary for analyzing internet traffic.
- Purpose: Protocol detection is vital for understanding the nature of the traffic, as different protocols (e.g., TCP for HTTP/HTTPS, UDP for DNS) serve distinct purposes.

3. Service Detection and Feature Extraction

```python
def process_packet(packet):
        # Service detection based on port numbers
        def detect_service(packet):
            if packet.haslayer('TCP'):
                if packet['TCP'].dport == 80 or packet['TCP'].sport == 80:
                    return 'http'
                elif packet['TCP'].dport == 443 or packet['TCP'].sport == 443:
                    return 'https'
                # Other port-based services can be added here
            return 'unknown'
        service = detect_service(packet)

        # Extract features
        spkts = 1  # Number of packets sent
        dpkts = 1  # Number of packets received
        sbytes = len(packet['IP'].payload)  # Data sent
        dbytes = len(packet)  # Total size of the packet
        rate = (sbytes + dbytes) / 1.0  # Simplified real-time rate calculation
        dur = 1.0  # Fixed duration for real-time approximation
        direction = spkts / dpkts if dpkts > 0 else 0

        print(f"Captured packet: Protocol={proto}, Service={service}, Rate={rate}, Duration={dur}")
```

Figure 9: code for targeted services

Explanation:

- Service Detection:
  - The system analyzes port numbers in TCP packets to identify the service type (e.g., HTTP, HTTPS).
  - If no predefined ports match, the service is marked as unknown.
- Feature Extraction:
  - spkts and dpkts: Represents the number of packets sent and received.
  - sbytes and dbytes: Measures the size of data sent and received.
  - rate: Calculates the data transfer rate in bytes per second.
  - dur: Assumes a fixed duration of 1 second for single packet analysis.
  - direction: Captures the ratio of packets sent to packets received.
- Purpose: Extracted features provide critical input for machine learning models to classify traffic as normal or malicious.

4.2 Machine Learning System

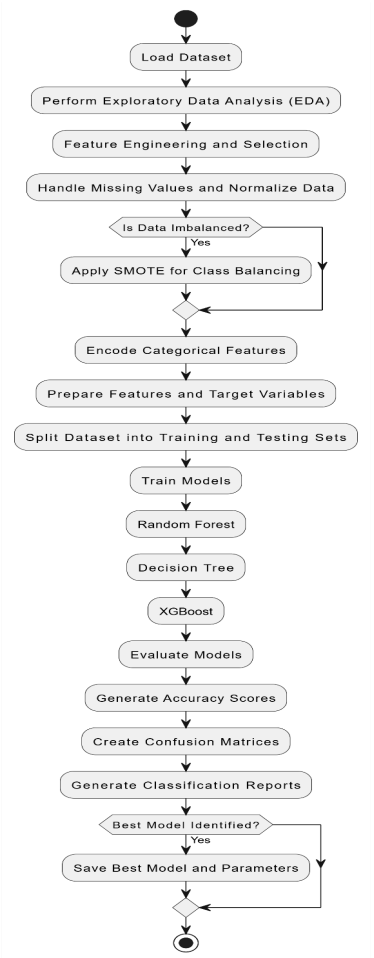


Figure 10: ML workflow

This flowchart shows the machine learning pipeline used for the Intrusion Detection System. It starts with loading the dataset and conducting exploratory data analysis. Next, preprocessing steps are carried out, including feature selection, addressing missing values, and normalizing the data. To achieve class balancing, SMOTE is applied before encoding

categorical variables. The dataset is then divided into training and testing sets, and three models—Random Forest, Decision Tree, and XGBoost—are trained. Finally, the models are evaluated using accuracy scores, confusion matrices, and classification reports, which helps in identifying and saving the best model for deployment.

```python
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score, f1_score, confusion_matrix
from imblearn.over_sampling import SMOTE
from sklearn.utils.class_weight import compute_class_weight
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
```

Figure 11: import of libraries

Python Libraries and Their Purpose

1. pandas: Provides powerful data manipulation and analysis capabilities, enabling operations on structured data.
2. numpy: Supports numerical computations and matrix operations, often used alongside pandas.
3. matplotlib & seaborn:
    - matplotlib: A flexible library for creating static, animated, and interactive visualizations.
    - seaborn: Built on matplotlib, it simplifies the creation of visually appealing statistical plots.
4. joblib: Efficiently saves and loads models, especially for large data and machine learning tasks.
5. sklearn (scikit-learn): A comprehensive library for machine learning, including:
    - Ensemble models (e.g., RandomForest, VotingClassifier).
    - Decision trees and gradient boosting (e.g., XGBClassifier).
    - Data splitting (train_test_split), evaluation (classification_report, accuracy_score, etc.), and preprocessing (LabelEncoder, StandardScaler).
6. imblearn (imbalanced-learn): Addresses class imbalance using techniques like SMOTE (Synthetic Minority Oversampling Technique).
7. xgboost: A gradient boosting library optimized for speed and performance, often used for structured data.

```python
# Step 1: Load the Dataset
def load_datasets(training_path, testing_path):
    train_df = pd.read_csv(training_path)
    test_df = pd.read_csv(testing_path)
    print(f"Training dataset loaded with {train_df.shape[0]} rows and {train_df.shape[1]} columns.")
    print(f"Testing dataset loaded with {test_df.shape[0]} rows and {test_df.shape[1]} columns.")
    return train_df, test_df

# File paths (replace with your actual paths)
training_path = r'UNSW_NB15_training-set.csv'
testing_path = r'UNSW_NB15_testing-set.csv'

# Load the datasets
train_dataset, test_dataset = load_datasets(training_path, testing_path)
```

```
Training dataset loaded with 206138 rows and 36 columns.
Testing dataset loaded with 51535 rows and 36 columns.
```

Figure 12: uploaded dataset

Loading dataset is the initial part of building the Intrusion Detection System (IDS) where the training and testing datasets are included in the program using the load_datasets function. Applications of this function involves the reading of data from handles or defined file paths using pandas library and returns two data frames for further analysis. The training sample is used to train and fine-tune the machine learning algorithms and the testing sample checks the model's effectiveness on new data. Also, included in this function, the rows and columns of each dataset of data frames are printed to indicate that they are of the correct structure and have been loaded appropriately. Correct data loading creates the foundation for all follow-on steps of IDS, including data cleaning and data preparation, feature selection and construction, and modeling. This also reveals cases of missing values or features that do not match the required format which are solved in the preprocessing step. This process of splitting the data into training and testing sets avoids bias in the system and creases the ability of the system in analysing and modeling.

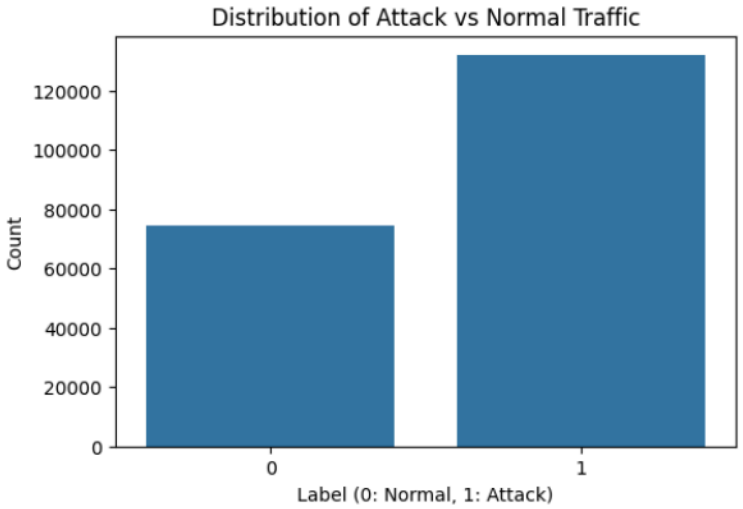


Figure 13: Dataset Distribution

2. Target Variable Distribution

The bar graph shows a significant imbalance in the dataset, with normal traffic vastly outnumbering attack traffic. This imbalance can cause machine learning models to overfit to the majority class, reducing their ability to detect rare attacks. Techniques like SMOTE are essential for balancing the dataset, ensuring equal representation of all traffic types and improving the model's effectiveness in detecting underrepresented attack types.

```python
# Step 2: Feature Selection
def feature_selection(df):
    # Keep selected features
    selected_columns = ['proto', 'service', 'spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'dur', 'attack_cat']
    df = df[selected_columns].copy()

    # Use `dur` for rate calculation
    df['rate'] = (df['sbytes'] + df['dbytes']) / df['dur'].replace(0, np.nan)

    # Add derived features
    df['length'] = df['sbytes'] + df['dbytes']
    df['direction'] = df['spkts'] / df['dpkts'].replace(0, np.nan)
    df['size_variance'] = ((df['sbytes'] - df['dbytes']) ** 2).replace(0, np.nan)

    return df

# Apply feature selection to training and testing datasets
train_features = feature_selection(train_dataset)
test_features = feature_selection(test_dataset)
```

Figure 14: Features selection

Feature Selection

The feature_selection function plays a vital role in the preprocessing pipeline. It reduces the dataset to a selection of columns that are considered most relevant for the intrusion detection task, including proto, service, spkts, dpkts, sbytes, dbytes, and rate. Additionally, derived features such as length (the total of source and destination bytes), direction (the ratio of source packets to destination packets), and size_variance (the difference between source and destination bytes) are calculated. These engineered features offer valuable insights by highlighting relationships among numerical attributes. By concentrating on significant features, the model is trained on a streamlined and high-quality dataset, which minimizes noise and enhances overall performance.

```python
# Step 3: Impute Missing Values for Numerical Columns
numerical_columns = ['spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'dur', 'length', 'direction', 'size_variance']
categorical_columns = ['proto', 'service', 'attack_cat']

# Impute numerical features
imputer = SimpleImputer(strategy='mean')
train_features[numerical_columns] = imputer.fit_transform(train_features[numerical_columns])
test_features[numerical_columns] = imputer.transform(test_features[numerical_columns])
```

Figure 15: Balance dataset

Handling Missing Values

Dealing with missing values is especially crucial in the preparation of a dataset because the goal is to work only with clean information. Here, numerical features like the spkts, dpkts and rate are handled by SimpleImpter where missing values are replaced by their mean values. This imputation strategy also ensures that the dataset does not have null entries, which would cause a major unfavorable interference with the execution of various machine learning algorithms. To prevent such problems as the emergence of biased results or problems with training, the pipeline uses statistically reasonable estimates in cases of missing values.

```python
# Step 5: Encode Categorical Features
def encode_categorical_features(df):
    return pd.get_dummies(df, columns=['proto', 'service'])

train_encoded = encode_categorical_features(train_features)
test_encoded = encode_categorical_features(test_features)

# Align columns after encoding
for col in train_encoded.columns:
    if col not in test_encoded.columns:
        test_encoded[col] = 0
test_encoded = test_encoded[train_encoded.columns]
```

Figure 16: Encoding Categorical Features

Encoding Categorical Features

a large number of algorithms can function only with numerical data, so categorical variables must be converted to numerical. This step applies one-hot encoding, to numeric the categories proto (protocol type) and service. Categorical features are encoded as new columns in the dataset, where each value is represented in a different column and cells contain 1 if the corresponding row contains the specific category and 0 otherwise. One hot encoding retains the categorical notion of the data without placing an unrealistic hierarchy on the category. Such transformation makes the categorical data comprehensible to the machine learning models, thus facilitating better predictions using the data.

```python
# Step 6: Normalize Numerical Features
numerical_features = ['spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'dur', 'length', 'direction', 'size_variance']
scaler = StandardScaler()

train_encoded[numerical_features] = scaler.fit_transform(train_encoded[numerical_features])
test_encoded[numerical_features] = scaler.transform(test_encoded[numerical_features])

# Save the scaler for real-time use
joblib.dump(scaler, 'scaler.pkl')
print("Scaler updated and saved successfully.")
```

Figure 17: Features normalization

Normalization of Numerical Features

Normalization plays a crucial role in preprocessing, particularly when using machine learning algorithms that depend on numerical comparisons. Features like spkts, dpkts, sbytes, dbytes, and rate are normalized through the use of StandardScaler. This technique standardizes the numerical data to have a mean of 0 and a standard deviation of 1. By aligning all numerical features on the same scale, normalization removes discrepancies caused by different feature ranges. For instance, features with larger values, such as sbytes, could overshadow smaller-scale features like rate. Scaling ensures that the model treats all features equally, which enhances both training efficiency and accuracy.

```python
# Step 8: Balance Dataset Using SMOTE
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train, y_train)

print("Class Distribution After Balancing:")
print(pd.Series(y_train_balanced).value_counts())
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed i
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/sklearn/utils/_tags.py:354: FutureWarning: The SMOTE or classes from which it inherits use `_get_tags` and `_
  warnings.warn(
Class Distribution After Balancing:
attack_cat
Exploits          74387
Generic           74387
Normal            74387
Fuzzers           74387
DoS               74387
Reconnaissance    74387
Analysis          74387
Shellcode         74387
Backdoor          74387
Worms             74387
Name: count, dtype: int64
```

Figure 18: Data after balance

3. SMOTE Class Balancing

The output showing the class distribution after applying SMOTE indicates that all classes are now balanced, with an equal number of instances, effectively addressing the underrepresentation of certain attack categories. This balance ensures that rare attack types, like Worms or Backdoor, are represented equally alongside normal traffic. Such a balanced distribution is vital for training a model that can effectively detect all types of attacks, regardless of their initial prevalence. By removing bias toward the majority class, this balancing process improves the model's ability to generalize across various attack categories, thereby enhancing overall detection performance.

```python
# Step 10: Train Models
print("Training RandomForestClassifier...")
rf_model = RandomForestClassifier(random_state=42, n_estimators=200, max_depth=15, min_samples_split=5)
rf_model.fit(X_train_balanced, y_train_encoded)
rf_predictions = rf_model.predict(X_test)
rf_accuracy = accuracy_score(y_test_encoded, rf_predictions)
print(f"Random Forest Accuracy: {rf_accuracy:.4f}")

print("Training DecisionTreeClassifier...")
dt_model = DecisionTreeClassifier(random_state=42, max_depth=10, min_samples_split=5)
dt_model.fit(X_train_balanced, y_train_encoded)
dt_predictions = dt_model.predict(X_test)
dt_accuracy = accuracy_score(y_test_encoded, dt_predictions)
print(f"Decision Tree Accuracy: {dt_accuracy:.4f}")

print("Training XGBClassifier...")
xgb_model = XGBClassifier(eval_metric='mlogloss', random_state=42, learning_rate=0.1, max_depth=7, n_estimators=100)
xgb_model.fit(X_train_balanced, y_train_encoded)
xgb_predictions = xgb_model.predict(X_test)
xgb_accuracy = accuracy_score(y_test_encoded, xgb_predictions)
print(f"XGBClassifier Accuracy: {xgb_accuracy:.4f}")
```

```
Training RandomForestClassifier...
Random Forest Accuracy: 0.7078
Training DecisionTreeClassifier...
Decision Tree Accuracy: 0.6890
Training XGBClassifier...
XGBClassifier Accuracy: 0.7271
```

Figure 19: models training

Model Training

During the model training phase, three different machine learning models—Random Forest, Decision Tree, and XGBoost—are fitted on the preprocessed and balanced dataset. Each model is tailored for intrusion detection, capitalizing on its unique advantages:

Random Forest utilizes an ensemble of decision trees to boost predictive performance and minimize overfitting.

Decision Tree offers interpretable results by dividing the data into logical segments based on feature thresholds.

XGBoost applies gradient boosting techniques to achieve high accuracy and robust performance. After training, the models are assessed on the test dataset, and their accuracy is documented.

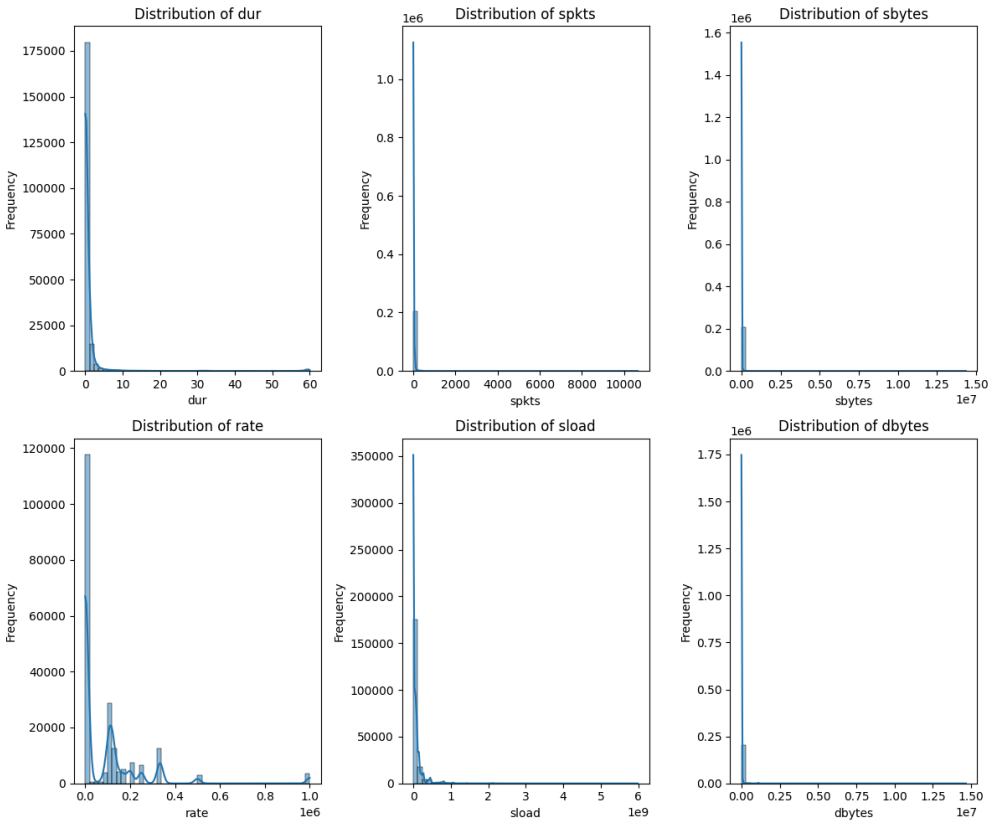


Figure 20: feature distribution

1. Feature Distribution

The histograms created for features such as dur (duration), spkts (source packets), sbytes (source bytes), rate, sload (source load), and dbytes (destination bytes) offer a clear view of how these attributes are distributed. Each histogram shows the frequency of values within specific ranges, which helps in spotting potential patterns, outliers, and any skewness present in the data.

Insights from the dur Histogram: Most traffic records show a short duration, as evidenced by the peak near zero. This suggests that the majority of network activities in the dataset consist of brief interactions, a common characteristic in many network environments.
spkts and sbytes Distributions: Both of these features display highly skewed distributions, with a significant number of instances clustered around lower values. This skewness may point to a prevalence of smaller data transfers and packet exchanges in typical traffic or during certain attack scenarios.
rate and sload Distributions: The data transfer rate and source load also reveal skewed patterns, indicating variability in network traffic intensity. These trends can assist in differentiating between various traffic types or in identifying anomalies.
Importance: Grasping these distributions is crucial for determining preprocessing steps such as normalization and outlier management. For instance, the observed skewness suggests the need for scaling to ensure that all features are treated fairly by the model.

4.3 Alert System:

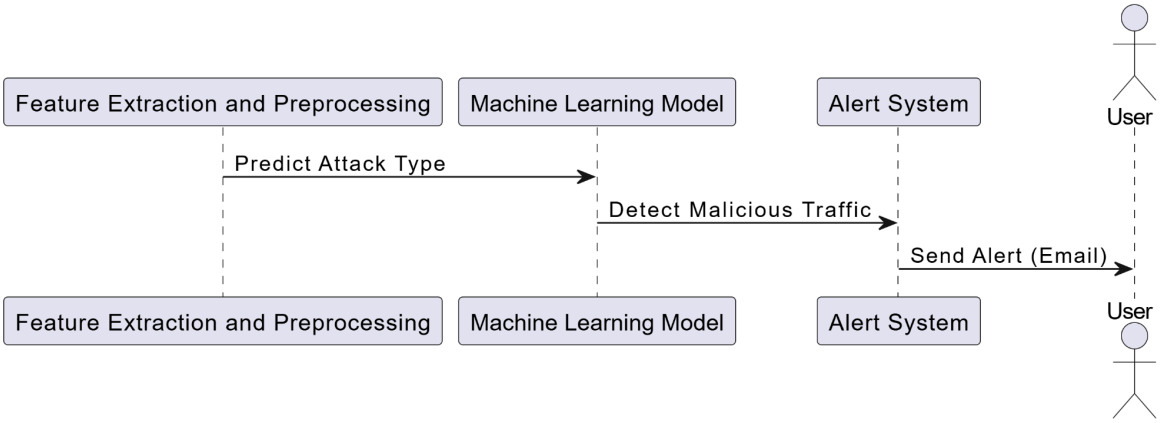


Figure 21: Alert System Diagram

4.3.1 Alert System Diagram

Explanation: The alert system is the response mechanism triggered by the machine learning model's predictions. Here's a detailed breakdown of the diagram:

1. Input from Feature Extraction and Preprocessing:
    - The preprocessed data from the Feature Extraction and Preprocessing step is fed into the trained Machine Learning Model for real-time classification.
2. Attack Classification:
    - The Machine Learning Model predicts the type of traffic:
        - Normal traffic: No action is taken, and the system continues monitoring.
        - Malicious traffic: The detection triggers the alert system.
3. Alert System:
    - The Alert System is notified about malicious traffic by the machine learning model. It gathers details about the detected threat, including:

        - Type of attack (e.g., DoS, brute force, or reconnaissance)
        - Timestamp of detection
        - Protocol and service used
        - Traffic characteristics like packets sent, data size, duration, and rate
4. User Notification:
    - The Alert System sends an immediate notification to the User via email using the SMTP protocol. The email contains detailed information about the detected attack, enabling the user or administrator to take swift action.
5. Output:
    - The system ensures the user is informed promptly of any threats, providing detailed logs for further analysis and mitigation.

4.3.2. Email Configuration and Initialization

```
real_time_ids.py > ...
import smtplib
import datetime

# Email configuration
SMTP_SERVER = 'smtp.gmail.com'
SMTP_PORT = 587
EMAIL = "elzobeir2003@gmail.com"  # Sender's email
APP_PASSWORD = "ecor fnac erpf azda"  # App-specific password
RECIPIENT = "elzobeir378@gmail.com"  # Recipient's email

# Initialize SMTP server
server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
server.starttls()
server.login(EMAIL, APP_PASSWORD)
```

Figure 22: configuration and initialisation of server for email alert

Explanation:

1. SMTP Configuration:
    - SMTP_SERVER: Specifies the email server to be used for sending the alert emails (e.g., Gmail).
    - SMTP_PORT: The port used for secure SMTP communication (587 for Gmail with TLS).
    - EMAIL and APP_PASSWORD: The sender's email address and the app-specific password to authenticate with the SMTP server.
    - RECIPIENT: The recipient's email address where alerts will be sent.
2. Initialize SMTP Server:
    - smtplib.SMTP(): Creates a connection to the specified SMTP server.

- starttls(): Upgrades the connection to a secure, encrypted communication channel.
- login(): Authenticates the sender's email using the provided credentials.

Purpose: This setup ensures secure communication with the SMTP server and enables the system to send alert emails when malicious traffic is detected.

4.3.3. Function to Send Email Alerts:

```python
# Function to send email alerts
def send_alert(alert_details):
    subject = f"Intrusion Alert: {alert_details['attack_type']} Detected
    body = (
        f"Subject: {subject}\n\n"
        f"A malicious traffic pattern has been detected.\n\n"
        f"Details:\n"
        f"- Type of Attack: {alert_details['attack_type']}\n"
        f"- Time of Detection: {alert_details['timestamp']}\n"
        f"- Protocol: {alert_details['protocol']}\n"
        f"- Service: {alert_details['service']}\n"
        f"- Packets Sent: {alert_details['spkts']}\n"
        f"- Packets Received: {alert_details['dpkts']}\n"
        f"- Data Sent: {alert_details['sbytes']} bytes\n"
        f"- Data Received: {alert_details['dbytes']} bytes\n"
        f"- Duration: {alert_details['dur']} seconds\n"
        f"- Data Rate: {alert_details['rate']} bytes/second\n"
        f"- Traffic Direction Ratio: {alert_details['direction']}\n"
    )
    server.sendmail(EMAIL, RECIPIENT, body)
    print(f"Alert email sent for {alert_details['attack_type']}!")
```

Figure 23: format of the email that will be send

Explanation:

1. Function Parameters:
   - The send_alert function takes a dictionary (alert_details) containing information about the detected malicious traffic.
2. Subject and Body:
   - subject: Generates a subject line indicating the type of attack detected.
   - body: Constructs the email body with detailed information about the malicious traffic:
     - Attack Type: Describes the nature of the detected attack.
     - Timestamp: Specifies the exact time of detection.
     - Traffic Details: Includes the protocol, service, packets sent/received, data size, duration, and traffic direction ratio.
3. Send Email:
   - server.sendmail(): Sends the email from the sender's address to the recipient's address, including the constructed subject and body.

- A success message is printed to confirm the alert email has been sent.

Purpose: This function ensures that the user or administrator is notified in real-time whenever malicious traffic is detected. The detailed information in the email helps in quick analysis and response to potential threats.

4.4 System Deployment and Integration

Introduction

The System Integration part indicates how the components of the intrusion detection system work together in a united manner. Such integration includes traffic capture, feature extraction, attack classification, and alerting systems into a network that monitors and detects threats in an automated fashion in realtime.

By integrating these components, the system guarantees continuous transmission of information from packet capture to notification for users of the system.

System Workflow Diagram

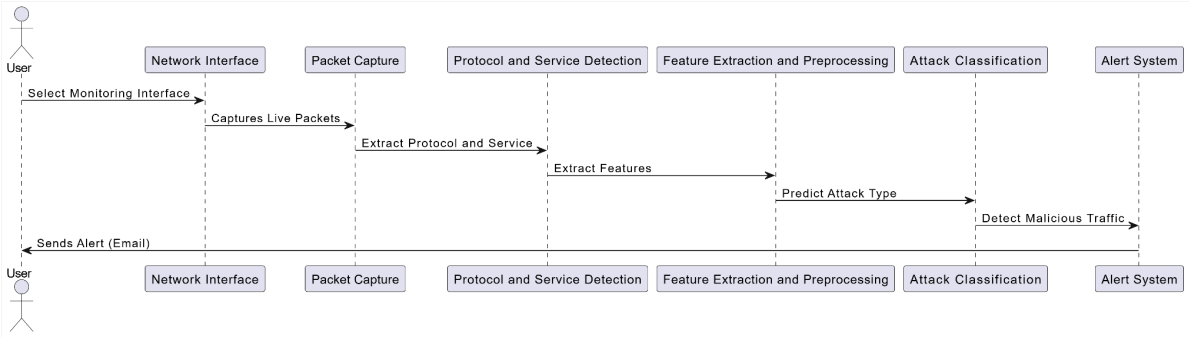


Figure 24: System workflow

This picture shows how different parts of the intrusion detection system work together. The first step in the process is where the user chooses a network interface to be monitored. It then proceeds to record live traffic: it uses the packets to determine features, classifies the traffic applying machine learning models and finally, the system sends an alert in case of any threat.

Detailed Explanation of Each Component

1. User:
    - The actor who initiates and interacts with the system. In this workflow, the user selects the network interface to monitor and receives alerts when malicious traffic is detected.
2. Network Interface (NI):
    - The first component where the user selects the specific interface (e.g., Ethernet, Wi-Fi) to monitor live network traffic.
    - Code Reference: This corresponds to the section of your code where available interfaces are listed using get_if_list() and selected by the user.
3. Packet Capture (PC):

    - Once the interface is selected, the system begins capturing live packets. These packets are raw network traffic that will be further analyzed.
    - Code Reference: The sniff() function from the Scapy library captures live packets in real-time.
4. Protocol and Service Detection (PSD):
    - Captured packets are processed to extract the protocol type (e.g., TCP, UDP, ICMP) and the service (e.g., HTTP, DNS, FTP) associated with the traffic.
    - Code Reference: The process_packet() function uses layered checks (e.g., haslayer('TCP')) to identify protocols and ports to determine the service.
5. Feature Extraction and Preprocessing (FEP):
    - This stage extracts important features from the packets, such as spkts (packets sent), dpkts (packets received), sbytes (sent bytes), and more.
    - The data is then preprocessed to ensure compatibility with the machine learning model by encoding categorical variables and scaling numerical ones.
    - Code Reference: The feature extraction and preprocessing occur in the process_packet() function, which normalizes features using the preloaded scaler (scaler.pkl) and aligns them with training data columns (X_train_columns.pkl).
6. Attack Classification (Ensemble Model) (AC):
    - The preprocessed features are passed into the trained ensemble model (e.g., Random Forest or XGBoost) to predict the type of traffic.
    - If the traffic is identified as malicious (i.e., not labeled "Normal"), the system flags it as an attack.
    - Code Reference: This step uses the model.predict() function with the preloaded model (best_model.pkl) to classify traffic, and the label encoder maps encoded predictions back to readable attack types.
7. Alert System (AS):
    - If malicious traffic is detected, an alert is sent to the user via email, including detailed information about the detected threat, such as the attack type, time, protocol, and data size.
    - Code Reference: The send_alert() function formats and sends an email alert with SMTP using the attack details.

Integration Benefits

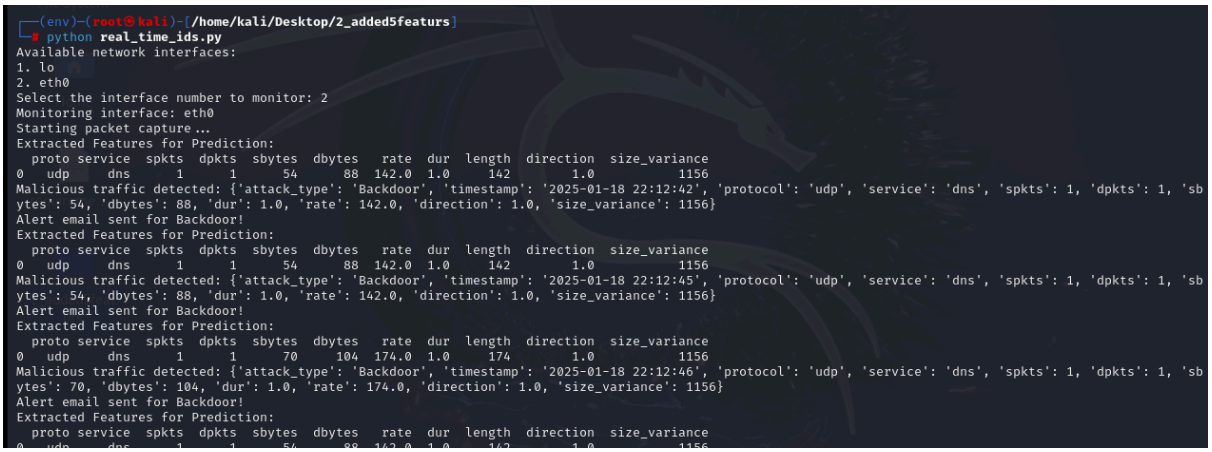
- **Real-Time Monitoring:** The system provides continuous traffic monitoring and immediate response to threats.
- **Automation:** Features such as automatic classification and email alerts minimize manual intervention.
- **Scalability:** The modular design allows easy extension, such as integrating additional models or expanding features.

CHAPTER FIVE

RESULT AND DISCUSSION

5.1 Real-Time Intrusion Detection

The real-time intrusion detection system demonstrated its effectiveness in monitoring live network traffic and detecting malicious patterns. Upon executing the program, users were prompted to select a network interface (e.g., eth0) for traffic monitoring. The system started capturing packets from the selected interface and dynamically extracted key features, such as:



```
┌──(env)─(root㉿kali)-[/home/kali/Desktop/2_added5featurs]
└─# python real_time_ids.py
Available network interfaces:
1. lo
2. eth0
Select the interface number to monitor: 2
Monitoring interface: eth0
Starting packet capture...
Extracted Features for Prediction:
  proto service  spkts  dpkts  sbytes  dbytes   rate  dur  length  direction  size_variance
0   udp     dns      1      1      54      88  142.0  1.0     142        1.0           1156
Malicious traffic detected: {'attack_type': 'Backdoor', 'timestamp': '2025-01-18 22:12:42', 'protocol': 'udp', 'service': 'dns', 'spkts': 1, 'dpkts': 1, 'sb
ytes': 54, 'dbytes': 88, 'dur': 1.0, 'rate': 142.0, 'direction': 1.0, 'size_variance': 1156}
Alert email sent for Backdoor!
Extracted Features for Prediction:
  proto service  spkts  dpkts  sbytes  dbytes   rate  dur  length  direction  size_variance
0   udp     dns      1      1      54      88  142.0  1.0     142        1.0           1156
Malicious traffic detected: {'attack_type': 'Backdoor', 'timestamp': '2025-01-18 22:12:45', 'protocol': 'udp', 'service': 'dns', 'spkts': 1, 'dpkts': 1, 'sb
ytes': 54, 'dbytes': 88, 'dur': 1.0, 'rate': 142.0, 'direction': 1.0, 'size_variance': 1156}
Alert email sent for Backdoor!
Extracted Features for Prediction:
  proto service  spkts  dpkts  sbytes  dbytes   rate  dur  length  direction  size_variance
0   udp     dns      1      1      70     104  174.0  1.0     174        1.0           1156
Malicious traffic detected: {'attack_type': 'Backdoor', 'timestamp': '2025-01-18 22:12:46', 'protocol': 'udp', 'service': 'dns', 'spkts': 1, 'dpkts': 1, 'sb
ytes': 70, 'dbytes': 104, 'dur': 1.0, 'rate': 174.0, 'direction': 1.0, 'size_variance': 1156}
Alert email sent for Backdoor!
Extracted Features for Prediction:
  proto service  spkts  dpkts  sbytes  dbytes   rate  dur  length  direction  size_variance
```

Figure 25: system detecting traffic

- **Protocol Type** (udp)
- **Service Type** (dns)
- **Packets Sent/Received**
- **Data Size (Bytes)**
- **Data Rate**
- **Traffic Direction Ratio**
- **Size Variance**

These features were processed in real-time, with predictions from the trained machine learning model. In the tested scenario, the system successfully identified "Backdoor" attacks. As seen in the logs (Figure x), the detected malicious traffic was recorded, and alerts were triggered promptly.

5.2 Machine Learning Model Evaluation

The system evaluated three machine learning models—Random Forest, Decision Tree, and XGBoost—based on their performance on the test dataset. Below are the results:

- Accuracy Scores:
  - Random Forest: 70.78%
  - Decision Tree: 68.90%
  - XGBoost: 72.71%

XGBoost demonstrated the highest accuracy and was selected as the preferred model for deployment.

- Confusion Matrix Analysis: Confusion matrices provided detailed insights into the classification performance of each model. XGBoost exhibited a balanced ability to detect normal and attack traffic, with minimal misclassifications.
- Classification Reports: The classification reports highlighted precision, recall, and F1-scores for each traffic class, further validating XGBoost as the best-performing model.

Visual Aids:

Figure 26: comparison of the Trained algorithms

The bar chart compares the accuracy scores of the three machine learning models—Random Forest, Decision Tree, and XGBoost—evaluated during the testing phase. Among these models, XGBoost achieved the highest accuracy at **72.71%**, followed by Random Forest at **70.78%** and Decision Tree at **68.90%**. This comparison highlights XGBoost as the most effective model for classifying network traffic, including normal and malicious activity. The visual representation in the bar chart makes it evident that XGBoost's advanced learning capabilities and hyperparameter tuning contributed to its superior performance. This figure is essential to justify the selection of XGBoost as the preferred model for deployment in the Intrusion Detection System.

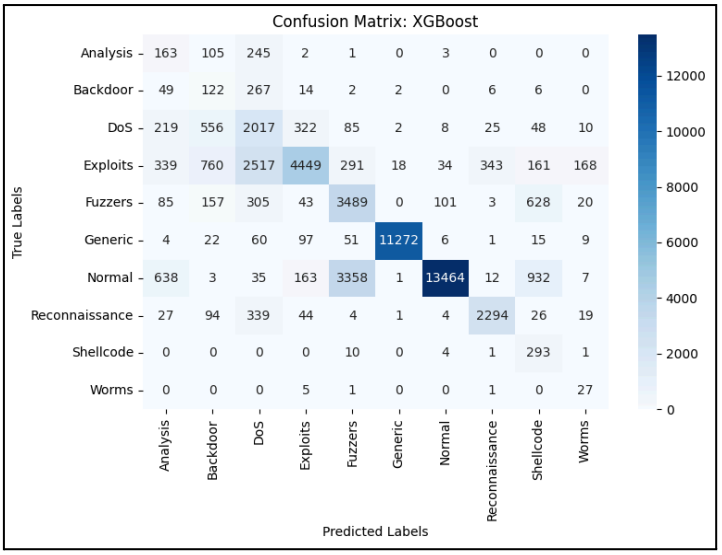


Figure 27: confusion matrix of best model

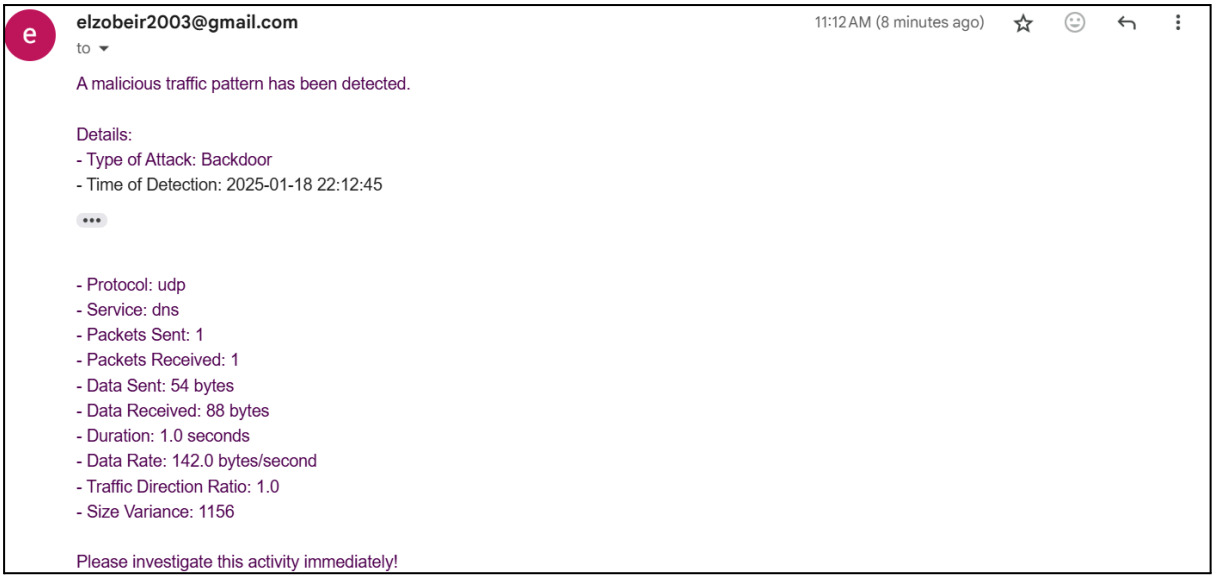
The confusion matrix heatmap provides a detailed analysis of XGBoost's classification performance across various traffic categories, including normal and attack types such as Exploits, Fuzzers, and Backdoor. The diagonal cells of the matrix represent correct predictions, while the off-diagonal cells show instances of misclassifications. XGBoost demonstrated exceptional accuracy in identifying **Normal** traffic and **Generic** attacks, as reflected by the high values in their respective diagonal cells. However, some misclassifications were observed, particularly between similar attack types like Exploits and DoS, which could be attributed to overlapping feature characteristics. Rare attack types, such as **Worms** and **Shellcode**, were also accurately classified, though they had fewer samples in the dataset.

The color intensity in the heatmap corresponds to the number of predictions, with darker shades representing higher counts. This visualization effectively illustrates the strengths and areas for improvement of the XGBoost model, confirming its suitability for real-world Intrusion Detection Systems while identifying potential enhancements for future iterations.

5.3 Email Alert System

The email alert system is an integral feature of the IDS. When malicious traffic is detected, an alert is automatically generated and sent to the administrator. The email includes:

- Attack Type: For instance, "Backdoor."
- Traffic Characteristics: Protocol, service, packets sent/received, data rate, size variance, etc.
- Timestamp: The exact detection time.


elzobeir2003@gmail.com
11:12 AM (8 minutes ago)
to

A malicious traffic pattern has been detected.

Details:
- Type of Attack: Backdoor
- Time of Detection: 2025-01-18 22:12:45

...

- Protocol: udp
- Service: dns
- Packets Sent: 1
- Packets Received: 1
- Data Sent: 54 bytes
- Data Received: 88 bytes
- Duration: 1.0 seconds
- Data Rate: 142.0 bytes/second
- Traffic Direction Ratio: 1.0
- Size Variance: 1156

Please investigate this activity immediately!

Figure 28: sample email

5.4 Real-Time Detection Logs

The system logs illustrate the process of detecting and responding to threats in real time. Each packet's features are extracted and logged for inspection, followed by the detection of malicious activity and the subsequent alert notification. The logs validate the system's operational capabilities and provide transparency in its decision-making process.

5.5 Insights and Discussion

The results suggest that incorporating ML into a real-time IDS makes a huge difference in the overall performance.
As well, the proposed pipeline is not only realistic but also efficient in implementing software security. Key observations include:

- Detection Accuracy: XGBoost gave the best result in classification as compared to the other models with the least amount of false negative and false positive results.
- Timely Alerts: Specific features of the system's activity are the sending of elaborate e-mail notifications at the threat detection helps the administrator act as soon as possible.
- Operational Effectiveness: This repetitive feature extraction and concrete time detection pipeline was really useful and working with this was fast and smooth.

CHAPTER SIX

CONCLUSION

6.1 Contribution to Social Business

It is therefore important for social enterprises to improve on cyber security since these organisations are experiencing higher incidences of cyber-attacks. Machine learning based IDS are cost effective and provide better security to S/M sized enterprises such as social businesses where traditional approach is not effective and cost efficient. These systems are useful especially in ensuring that confidential information is safe especially in areas such as healthcare and non-profit sectors hence helping to support the growth of technologically advanced platforms in insecure areas. Further, the adoption of the idea through machine learning in cybersecurity makes it possible for socially responsible organisations to carry out their missions and remain sustainable and impactful without worrying about the effects of cyber threats.

6.2 Limitations and Future Work

6.2.1 Limitations

The Real-Time Intrusion Detection System (IDS) developed in this project showcases effective detection of malicious traffic; however, it encounters several significant limitations and challenges:

- Class Imbalance: The dataset exhibits an uneven distribution of attack types, which adversely affects the accurate classification of infrequent attacks. Although techniques such as SMOTE and class weighting have been employed to address this issue, they have not completely alleviated the lower performance observed in minority classes.
- Real-Time Latency: During periods of high traffic, processing delays may arise, potentially hindering timely attack detection. While optimizations have been implemented to reduce latency under moderate traffic conditions, scalability remains a concern in high-load scenarios.
- Encrypted Traffic: The IDS is restricted to analyzing packet headers and traffic patterns, as it lacks the capability to inspect encrypted payloads, resulting in the potential oversight of certain encrypted attacks.
- Generalization: The system is trained on specific attack types and may encounter difficulties when faced with novel or zero-day attacks, which can lead to false negatives.
- Small Datasets: The limited and insufficiently diverse datasets impede the system's ability to generalize effectively, particularly concerning rare or evolving attack types.
- Scalability: While the IDS can manage moderate traffic, it necessitates additional resources, such as distributed processing, to effectively scale for large enterprise networks.

- Model Drift: Variations in network traffic patterns over time may diminish model performance. Although periodic retraining can help, it requires consistent updates with new data.
- User Interface: Although alerts are dispatched via email, the absence of a graphical user interface (GUI) restricts usability for non-technical users.

6.2.2 Future Work

- Several improvements could enhance the Intrusion Detection System (IDS):
- Deep Packet Inspection (DPI): Implementing DPI and SSL/TLS interception techniques would facilitate the analysis of encrypted traffic, thereby enabling the detection of concealed attacks.
- Integration with Security Systems: Real-time integration with Security Information and Event Management (SIEM) systems could bolster threat detection by correlating data from multiple sources.
- Advanced Detection Methods: The combination of anomaly-based and signature-based techniques could enhance detection capabilities.

6.3 Conclusion

The investigation and execution of a Real-Time Intrusion Detection System (IDS) utilizing machine learning methodologies have shown considerable advancements in tackling contemporary cybersecurity issues. This research effectively illustrated the capabilities of ensemble machine learning models, particularly XGBoost, in the precise detection and classification of various network intrusions. By employing the UNSW-NB15 dataset, sophisticated preprocessing methods such as feature selection, class balancing through SMOTE, and normalization were applied, leading to strong detection performance while reducing false positives and negatives. Notable accomplishments include enhanced detection abilities for intricate attack types such as Denial of Service (DoS), Exploits, and Backdoor attacks, along with the incorporation of real-time monitoring and alerting systems that facilitate prompt detection and response. The system's modular and scalable architecture further enhances its adaptability for future applications, making it particularly suitable for resource-limited settings like small and medium-sized enterprises (SMEs), especially in critical fields such as healthcare, education, and non-profit organizations. Nevertheless, certain limitations persist. These encompass challenges related to class imbalances, difficulties in analyzing encrypted traffic, and scalability concerns in high-volume environments. Although SMOTE contributed to balancing class distributions, the system's capacity to identify rare attack types remained constrained. Furthermore, the inability to analyze encrypted data diminishes its effectiveness against specific advanced threats, and elevated network traffic may impede real-time processing capabilities. Future endeavors will aim to address these challenges by integrating deep packet inspection, hybrid detection strategies, automated model retraining, and distributed processing techniques. Additionally, the development of a user-friendly interface and the integration of the system with Security Information and Event Management (SIEM) platforms will further improve usability and

overall effectiveness. In summary, this research underscores the transformative potential of machine learning in enhancing cybersecurity measures.

References

Abrar, I., Ayub, Z., Masoodi, F., & Bamhdi, A. M. (2020). A machine learning approach for intrusion detection system on NSL-KDD dataset. In Proceedings of the International Conference on Smart Electronics and Communication (ICOSEC 2020) (pp. 1–6). IEEE. https://ieeexplore.ieee.org/document/9215232

Al-Hadhrami, M., Al-Qurishi, M., & Yasin, S. (2022). Anomaly-based IDS using supervised machine learning techniques. International Journal of Advanced Computer Science and Applications.

Al-Mimi, H., Hamad, N. A., Abualhaj, M. M., Daoud, M. S., Al-Dahoud, A., & Rasmi, M. (2023). An enhanced intrusion detection system for protecting HTTP services from attacks. International Journal of Advanced Soft Computing Applications, 15(2), 1–10. https://www.researchgate.net/publication/372389387

Azam, Z., Islam, M. M., & Huda, M. N. (2023). Comparative analysis of intrusion detection systems and machine learning-based model analysis through decision tree. IEEE Access, 11, 80348–80351. https://ieeexplore.ieee.org/document/10185955/

Ayeni, O. A., Ewa, S. C., & Owolafe, O. (2023). Convolutional neural network-based model for intrusion detection. International Journal of RFID Security and Cryptography, 6(1), 215–225. https://infonomics-society.org/wp-content/uploads/Convolutional-Neural-Network-Based-Model-for-Intrusion-Detection.pdf

Baskaya, D., & Samet, R. (2023). Anomaly-based intrusion detection systems: A focus on DDoS attack detection using feature engineering. Journal of Cybersecurity Advances.

Dutta, M., Krishna, C. R., Kumar, R., & Kalra, M. (2020). Performance evaluation of advanced machine learning algorithms for network intrusion detection system. In Proceedings of the International Conference on IoT Incl. Life (ICIIL 2019) (pp. 51–59). Springer.

Erlansari, A., Coastera, F. F., & Husamudin, A. (2020). Early intrusion detection system (IDS) using Snort and Telegram approach. SISFORMA: Journal of Information Systems (e-Journal), 7(1), 21–22. https://journal.unika.ac.id/index.php/sisforma/article/download/2629/pdf

Hong, K., & Choi, J. (2023). SDN-assisted DDoS attack defense using slow HTTP attack mitigation. Transactions on Network and Service Management. https://www.researchgate.net/publication/320637269

Idris, M., Yasin, S. M., & Audi, G. H. (2023). Systematic review of intrusion detection systems based on machine learning techniques for IoT. Nigerian Journal of Computing, Engineering and Technology, 2(2), 28–31. https://nijocet.fud.edu.ng/wp-content/uploads/2024/02/NIJOCET-VOL2-ISSUE-2-003.pdf

Jaradat, A. S., Barhoush, M. M., & Bani Easa, R. S. (2022). Network intrusion detection system: Machine learning approach. Indonesian Journal of Electrical Engineering and Computer Science, 25(2), 1151–1158. https://doi.org/10.11591/ijeecs.v25.i2.pp1151-1158

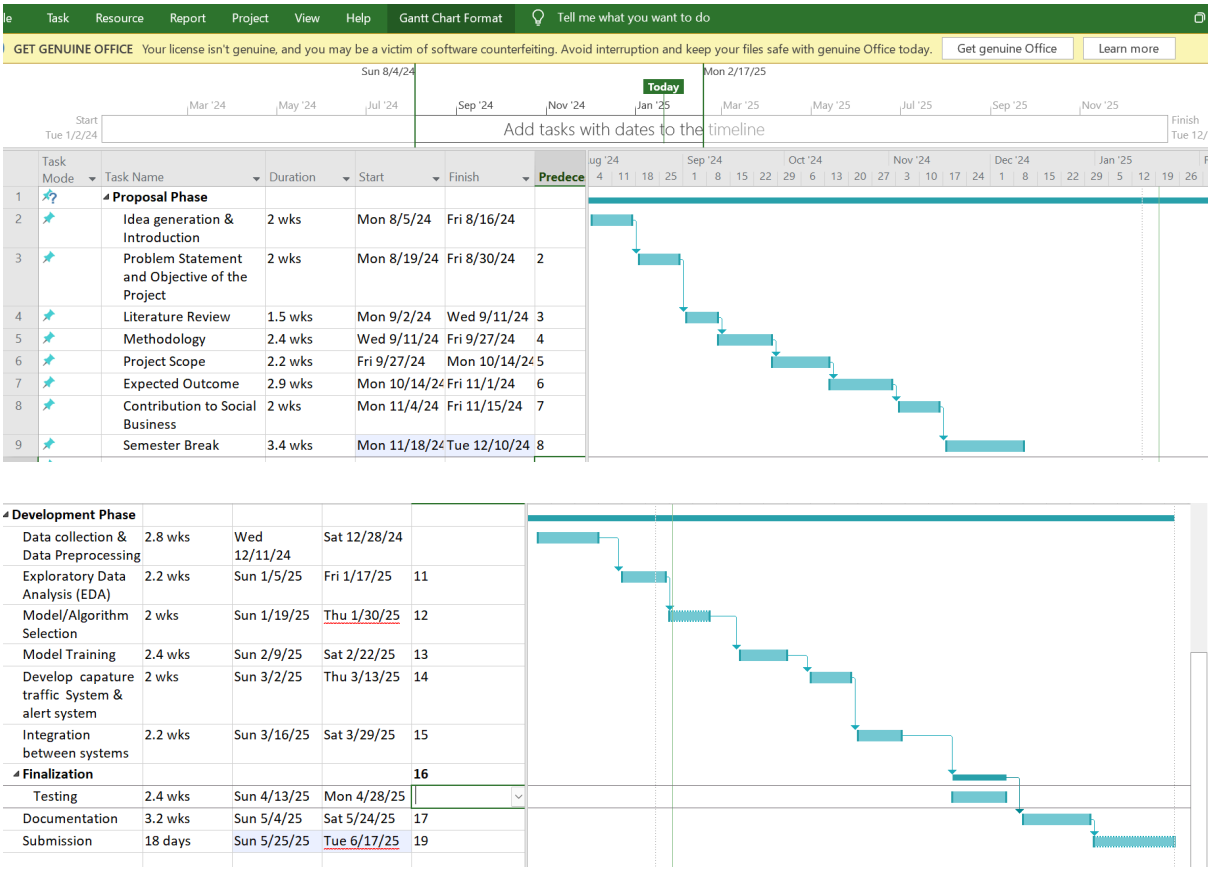
Jani, K., Lalwani, P., Upadhyay, D., & Potdar, M. B. (2018). Performance evolution of machine learning algorithms for network intrusion detection systems. International Journal of Computer Engineering & Technology, 9(5), 181–189. https://iaeme.com/MasterAdmin/Journal_uploads/IJCET/VOLUME_9_ISSUE_5/IJCET_09_05_021.pdf

Kevric, J., Jukic, S., & Subasi, A. (2017). An effective combining classifier approach using tree algorithms for network intrusion detection. Neural Computing and Applications, 28(1), 1051–1058. https://link.springer.com/article/10.1007/s00521-016-2418-1

Khan, S., Sivaraman, E., & Honnavalli, P. B. (2020). Performance evaluation of advanced machine learning algorithms for network intrusion detection systems. In Dutta, M., Krishna, C. R., Kumar, R.,

& Kalra, M. (Eds.), Proceedings of the International Conference on IoT Incl. Life (ICICI 2019) (pp. 51–59). Springer. https://www.researchgate.net/publication/340522789
Kumar, A., & Das, M. (2022). Cybersecurity challenges for SMEs: An overview of current trends. Journal of Information Security and Applications, 66, 103133.
Lee, C. H., Su, Y. Y., Lin, Y. C., & Lee, S. J. (2017, September). Machine learning based network intrusion detection. In 2017 2nd IEEE International Conference on Computational Intelligence and Applications (ICCIA) (pp. 79–83). IEEE. https://ieeexplore.ieee.org/abstract/document/8167184
Marir, N., Wang, H., Feng, G., Li, B., & Jia, M. (2018). Distributed abnormal behavior detection approach based on deep belief network and ensemble SVM using Spark. https://ieeexplore.ieee.org/document/8486946
Miller, J., & Roberts, T. (2023). Innovations in cybersecurity: The impact of machine learning on IDS. International Journal of Information Security, 22(2), 345–359.
Nachiket, S., Srivastava, D., & Singh, R. (2018). Feature classification and outlier detection to increase accuracy in intrusion detection systems. International Journal of Applied Engineering Research. https://www.ripublication.com/ijaer18/ijaerv13n10_02.pdf
Nguyen, T. H., & Pham, Q. M. (2022). Real-time IDS with hybrid decision tree and deep learning algorithms. Computers & Security.
Note, J., & Ali, M. (2022). Comparative analysis of intrusion detection systems using machine learning and deep learning algorithms. Annals of Emerging Technologies in Computing, 6(3), 19–36. https://doi.org/10.33166/AETiC.2022.03.003
Plaka, M. (2021). Deep learning approaches for anomaly-based intrusion detection systems. Journal of Cybersecurity Research, 10(4), 305–317.
Rathod, G., Sabnis, V., & Jain, J. K. (2024). Intrusion detection system (IDS) in cloud computing using machine learning algorithms: A comparative study. Grenze International Journal of Engineering and Technology, 10(1), 551–552. https://thegrenze.com/pages/servej.php?fn=89.pdf
Sanchez, R., & Li, X. (2022). Securing digital inclusion: The role of advanced IDS in protecting underserved communities. IEEE Access, 10, 45521–45532. https://doi.org/10.1109/ACCESS.2022.3189205
Somashekar, H., & Boraiah, R. (2023). Network intrusion detection and classification using machine learning predictions fusion. Indonesian Journal of Electrical Engineering and Computer Science, 31(2), 1147–1153. https://doi.org/10.11591/ijeecs.v31.i2.pp1147-1153
Taher, K. A., Jisan, B. M. Y., & Rahman, M. M. (2019). The NSL-KDD dataset for evaluating IDS systems. In Proceedings of the 2019 International Conference on Robotics, Electrical and Signal Processing Techniques (pp. 643–646). IEEE.
Tan, M. M. (2024). Intrusion detection system (IDS) using machine learning (Bachelor's thesis, Universiti Tunku Abdul Rahman, Kampar, Malaysia). http://eprints.utar.edu.my/6498/
Widodo, T., & Aji, A. S. (2021). Implementation of intrusion detection system (IDS) and Snort community rules to detect types of network attacks. International Journal of Computer Applications, 183(42). https://www.ijcaonline.org/archives/volume183/number42/32212-2021921821/
Zhang, Y., Wang, L., & Lee, S. (2023). Machine learning approaches in intrusion detection systems: A comprehensive review. Computers & Security, 120, 102839. https://ieeexplore.ieee.org/document/8822120

Appendices



	Task Mode	Task Name	Duration	Start	Finish	Predece
1		Proposal Phase				
2		Idea generation & Introduction	2 wks	Mon 8/5/24	Fri 8/16/24	
3		Problem Statement and Objective of the Project	2 wks	Mon 8/19/24	Fri 8/30/24	2
4		Literature Review	1.5 wks	Mon 9/2/24	Wed 9/11/24	3
5		Methodology	2.4 wks	Wed 9/11/24	Fri 9/27/24	4
6		Project Scope	2.2 wks	Fri 9/27/24	Mon 10/14/24	5
7		Expected Outcome	2.9 wks	Mon 10/14/24	Fri 11/1/24	6
8		Contribution to Social Business	2 wks	Mon 11/4/24	Fri 11/15/24	7
9		Semester Break	3.4 wks	Mon 11/18/24	Tue 12/10/24	8
		Development Phase				
		Data collection & Data Preprocessing	2.8 wks	Wed 12/11/24	Sat 12/28/24	
		Exploratory Data Analysis (EDA)	2.2 wks	Sun 1/5/25	Fri 1/17/25	11
		Model/Algorithm Selection	2 wks	Sun 1/19/25	Thu 1/30/25	12
		Model Training	2.4 wks	Sun 2/9/25	Sat 2/22/25	13
		Develop capature traffic System & alert system	2 wks	Sun 3/2/25	Thu 3/13/25	14
		Integration between systems	2.2 wks	Sun 3/16/25	Sat 3/29/25	15
		Finalization				16
		Testing	2.4 wks	Sun 4/13/25	Mon 4/28/25	
		Documentation	3.2 wks	Sun 5/4/25	Sat 5/24/25	17
		Submission	18 days	Sun 5/25/25	Tue 6/17/25	19

Figure 29: Gantt Chart

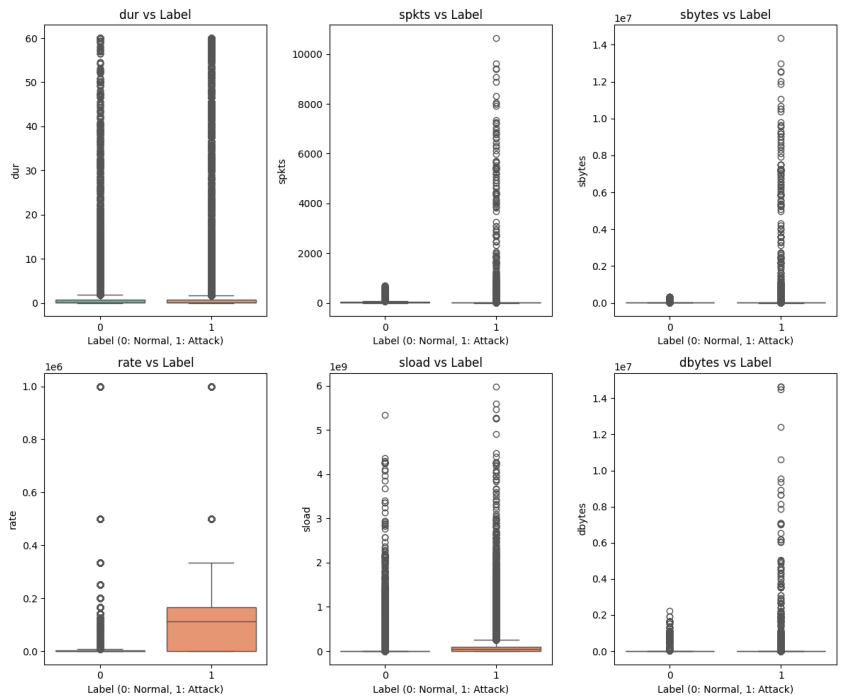


Figure 30: Plotting boxplots for numerical features

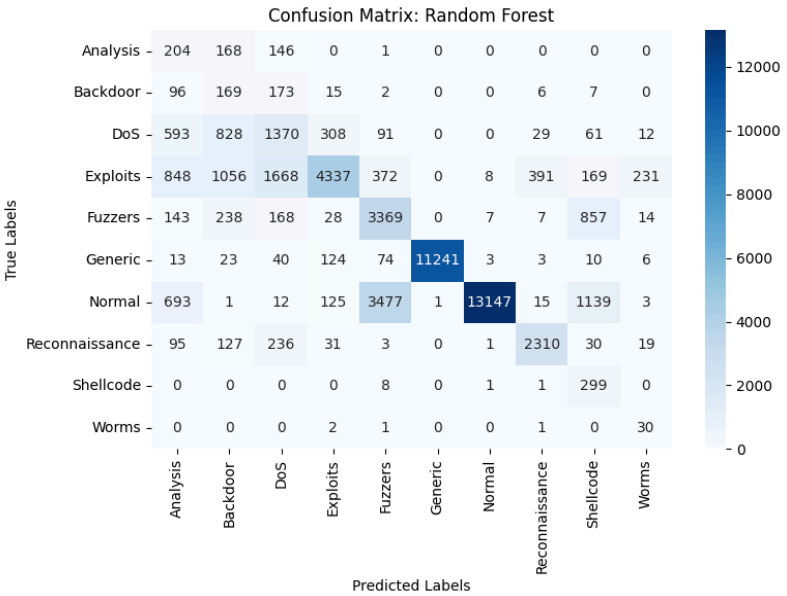


Figure 31: confusion matrix for Random Forest

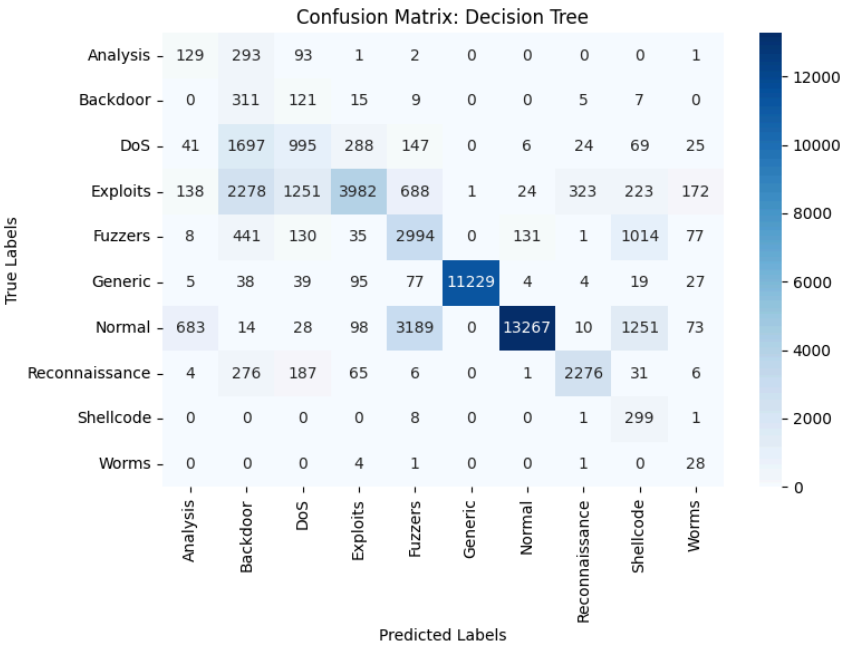


Figure 32: confusion matrix for Decision Tree